ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN – TIN HỌC

BÁO CÁO THỰC HÀNH

STLab02

Test Case Execution

Kiểm thử dựa trên yêu cầu với EduCommerce

Họ và tên	Đỗ Thành Vinh
Mã số sinh viên	20110112
Lớp	23TTH
Môn học	Kiểm thử phần mềm
GitHub	https://github.com/vinhdo1706/ST-Lab02
Phạm vi	REQ-02, REQ-07, REQ-09, REQ-10, REQ-15

Thành phố Hồ Chí Minh, ngày 28 tháng 10 năm 2026

Mục lục

1. Thông tin bài thực hành và phạm vi **5**

Phương pháp và bộ sản phẩm 5

2. Ứng dụng EduCommerce và các yêu cầu được chọn **7**

Phạm vi kiểm thử và lưu ý 7

3. Phân tích yêu cầu **8**

REQ-02 — Giỏ hàng 8

REQ-07 — Mã giảm giá 8

REQ-09 — Phí vận chuyển 8

REQ-10 — Thanh toán 8

REQ-15 — Tính nguyên tử khi trừ tồn kho 8

4. Điều kiện, tình huống kiểm thử và ma trận truy vết **9**

5. Các ca kiểm thử chi tiết và kết quả kỳ vọng **11**

Cấu trúc dữ liệu kiểm thử 11

Nhóm REQ-02 — Số lượng nguyên dương và cộng dồn sản phẩm trùng mã. 11

Nhóm REQ-07 — Kiểm tra tính hợp lệ của mã giảm giá và thời điểm ghi nhận đã sử dụng. 12

Nhóm REQ-09 — Phí vận chuyển nội thành và ngoại thành tại các mức giá biên. 14

Nhóm REQ-10 — Trạng thái đơn hàng và giao dịch thanh toán được duyệt hoặc từ chối. 15

Nhóm REQ-15 — Từ chối toàn bộ đơn hàng nếu có sản phẩm không đủ tồn kho. 17

6. Cấu trúc chương trình chạy kiểm thử bằng Python **19**

Dữ liệu khởi tạo và tính độc lập của ca kiểm thử 19

Đọc dữ liệu CSV và xử lý JSON 19

Điều phối yêu cầu và đối chiếu kết quả 19

Phân loại trạng thái và xuất báo cáo 20

7. Thực thi trên Google Colab **25**

8. Báo cáo kết quả, bảng tổng hợp và độ bao phủ **27**

Độ bao phủ và tính truy vết 27

9. Phân tích lỗi và những rủi ro còn lại **29**

10. Giải quyết tám tình huống thực tế **30**

11. Câu hỏi lý thuyết và bài tập phân tích mã **34**

Bài tập đọc và phân tích mã nguồn **36**

12. Tự đánh giá và tài liệu tham chiếu **38**

Kho mã nguồn và hướng dẫn chạy lại 38

Tài liệu tham chiếu 38

Kết luận **40**

1. Thông tin bài thực hành và phạm vi

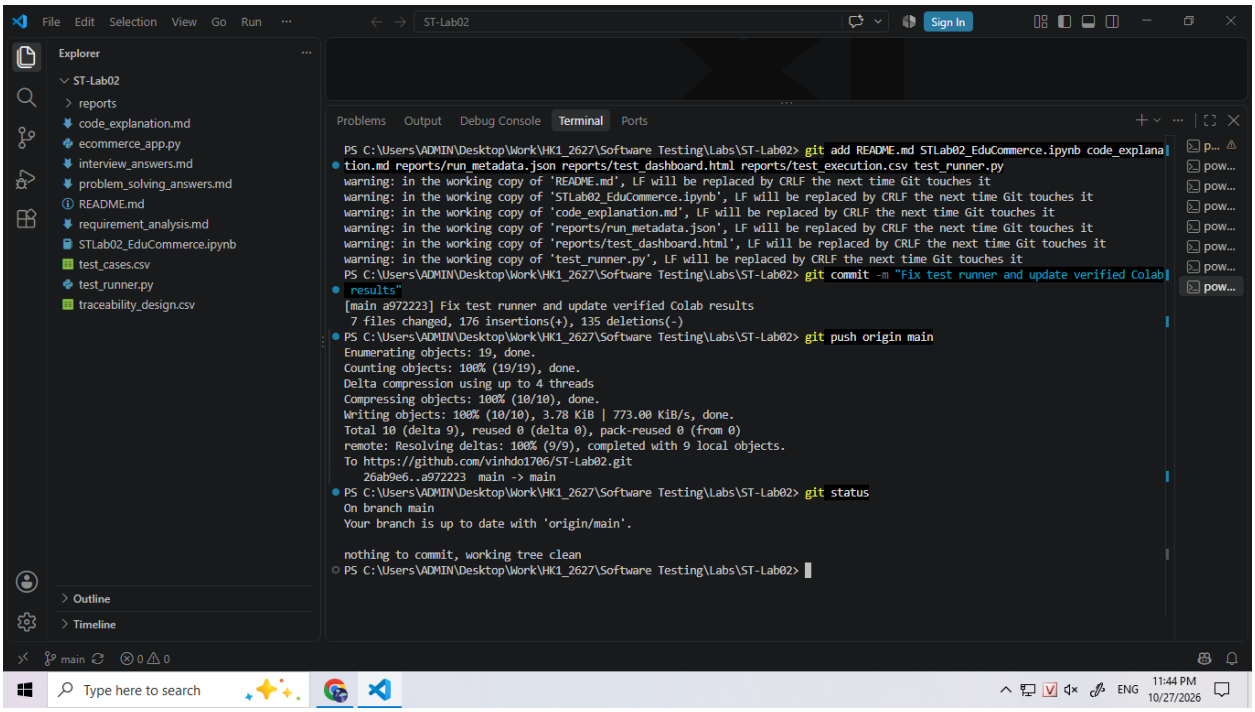
Đề tài triển khai kiểm thử dựa trên yêu cầu đối với ứng dụng EduCommerce: chọn năm yêu cầu chưa được làm mẫu, thiết kế 33 ca kiểm thử dựa trên dữ liệu, thực thi bằng Python trên Google Colab và xuất kết quả CSV cùng bảng tổng hợp HTML. Kết quả được xác minh tại thời điểm 27/10/2026, 23:27:02 (UTC+7): 33 PASS, 0 FAIL, 0 BLOCKED. Tỷ lệ PASS phản ánh bộ ca kiểm thử đã chọn, không có nghĩa toàn bộ ứng dụng không còn lỗi.

Thông tin	Nội dung
Môn học	Kiểm thử phần mềm
Tên bài	STLab02 — Test Case Execution
Họ tên / MSSV / lớp	Đỗ Thành Vinh / 20110112 / 23TTH
Ứng dụng kiểm thử	EduCommerce (mã nguồn trong tài liệu hướng dẫn, mục 2.6)
Yêu cầu đã chọn	REQ-02, REQ-07, REQ-09, REQ-10, REQ-15
Môi trường	Python 3.13.16 trong Google Colab; dữ liệu CSV; HTML bảng tổng hợp
GitHub	`https://github.com/vinhdo1706/ST-Lab02`
Phạm vi loại trừ	REQ-06, REQ-08, REQ-11 đã có ví dụ minh họa trong tài liệu hướng dẫn.

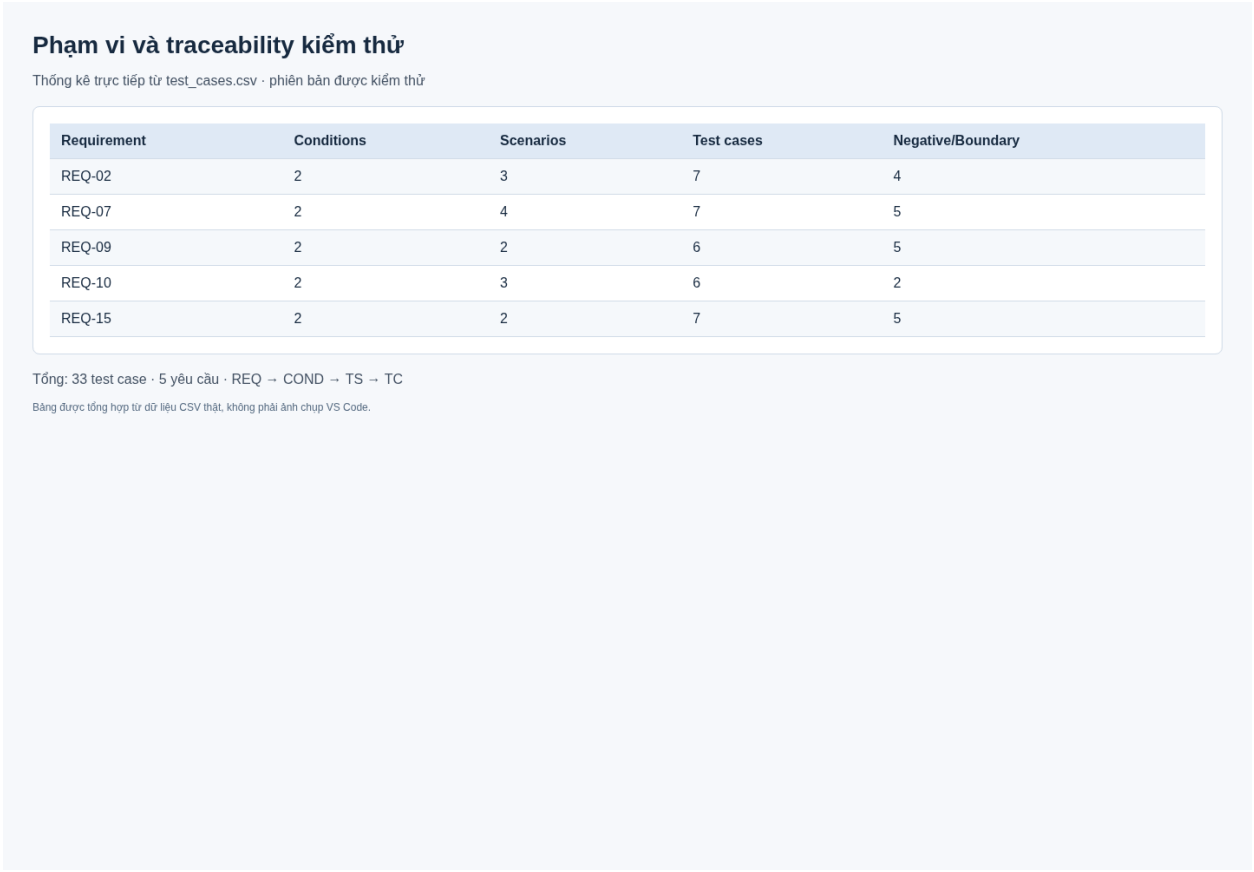
Phương pháp và bộ sản phẩm

Quy trình truy vết: yêu cầu (REQ) → điều kiện (COND) → tình huống (TS) → ca kiểm thử (TC) → lần chạy (RUN). Mỗi ca có mã riêng, dữ liệu chuẩn bị, giá trị đầu vào và kết quả kỳ vọng. Chương trình ghi kết quả thực tế vào tệp báo cáo riêng, không ghi đè dữ liệu gốc.

Tệp / thành phần	Vai trò
`ecommerce_app.py`	Ứng dụng gốc, giữ nguyên logic nghiệp vụ
`test_runner.py`	Đọc CSV, điều phối, so kỳ vọng/thực tế, xuất báo cáo
`test_cases.csv`	33 ca kiểm thử với điều kiện trước, steps, đầu vào và kỳ vọng
`traceability_design.csv`	Liên kết REQ → COND → TS → TC
`STLab02_EduCommerce.ipynb`	Notebook và output thực thi Google Colab
`reports/test_execution.csv`	Kết quả chi tiết từng ca kiểm thử
`reports/test_summary.csv`	Tổng hợp trạng thái theo REQ
`reports/traceability_execution.csv`	Truy vết đến RUN và trạng thái
`reports/test_dashboard.html`	Bảng tổng hợp HTML biểu đồ và bảng ca kiểm thử
`README.md`	Hướng dẫn tái hiện dự án



Hình 1: **E01 — Cấu trúc project trong VS Code** — Ảnh gốc phản ánh mã nguồn, runner, CSV và notebook.



Phạm vi và traceability kiểm thử

Thống kê trực tiếp từ test_cases.csv · phiên bản được kiểm thử

Requirement	Conditions	Scenarios	Test cases	Negative/Boundary
REQ-02	2	3	7	4
REQ-07	2	4	7	5
REQ-09	2	2	6	5
REQ-10	2	3	6	2
REQ-15	2	2	7	5

Tổng: 33 test case · 5 yêu cầu · REQ → COND → TS → TC

Bảng được tổng hợp từ dữ liệu CSV thật, không phải ảnh chụp VS Code.

Hình 2: **E02 — Tổng quan độ bao phủ của 5 yêu cầu** — Biểu diễn từ dữ liệu thiết kế của project.

2. Ứng dụng EduCommerce và các yêu cầu được chọn

EduCommerce là ứng dụng thương mại điện tử mô phỏng trong bộ nhớ, gồm các thành phần Catalog, Cart, Promotions, Shipping, Inventory, Payments, Checkout. Các ca kiểm thử gọi hàm nghiệp vụ trực tiếp. Mô hình không có giao diện người dùng hay cơ sở dữ liệu thật; vì vậy không suy diễn kết quả thành kiểm thử xuyên suốt toàn hệ thống.

Mã yêu cầu	Phân hệ	Tiêu chí chấp nhận
`REQ-02`	Cart	quantity là số nguyên dương; cùng SKU cộng dồn số lượng.
`REQ-07`	Promotion	Mã giảm giá không hợp lệ/đã dùng phải bị từ chối; chỉ đánh dấu used sau đơn thành công.
`REQ-09`	Shipping	Nội thành: subtotal $\geq$ 500.000 miễn phí; dưới ngưỡng 30.000; ngoại thành 50.000.
`REQ-10`	Payments	COD = PENDING_COD; ONLINE = PAID khi approved; declined không tạo đơn.
`REQ-15`	Inventory Atomicity	Một SKU thiếu tồn kho thì reject cả đơn, không trừ tồn kho SKU khác.

Phạm vi kiểm thử và lưu ý

Năm yêu cầu được chọn bao phủ giỏ hàng, khuyến mãi, vận chuyển, thanh toán và tồn kho. Giá trị kỳ vọng được xác định từ các tiêu chí chấp nhận. Những vấn đề chưa có quy tắc rõ, chẳng hạn thời điểm chốt giá sản phẩm, được nêu như giới hạn chứ không tự kết luận là lỗi ứng dụng.

3. Phân tích yêu cầu

REQ-02 — Giỏ hàng

Tác nhân: Khách hàng. Điều kiện trước: Cart gắn Catalog hợp lệ. Đầu vào: các thao tác add(SKU, quantity). Căn cứ xác định kết quả đúng: CartLine, số lượng cộng dồn và subtotal hoặc INVALID_QUANTITY. Rủi ro: chấp nhận quantity không nguyên dương và gây thay đổi giỏ không hợp lệ.

Điểm cần làm rõ / giới hạn: Giá trị bool được Python xem là int; phạm vi hiện tại chưa xác định quy tắc cho True/False nên không tự thêm căn cứ xác định kết quả đúng.

REQ-07 — Mã giảm giá

Tác nhân: Khách hàng. Điều kiện trước: mã giảm giá và giỏ được thiết lập. Đầu vào: mã giảm giá, used_codes và kết quả thanh toán. Căn cứ xác định kết quả đúng: INVALID_VOUCHER/VOUCHER_USED hoặc used_codes chỉ được cập nhật sau order thành công. Rủi ro: mất lượt dùng mã khi thanh toán bị từ chối.

Điểm cần làm rõ / giới hạn: Chính sách ngưỡng/giới hạn giảm giá SAVE10 của REQ-06 không được coi là nhiệm vụ kiểm thử mới.

REQ-09 — Phí vận chuyển

Tác nhân: Khách hàng. Đầu vào: subtotal (Decimal), vùng nhận hàng inner/outer. Căn cứ xác định kết quả đúng: 499.999 → 30.000; 500.000 → 0; 500.001 → 0 (inner). Outer → 50.000. Rủi ro: sai điều kiện biên hoặc sai vùng.

Điểm cần làm rõ / giới hạn: Quy tắc cho area ngoài inner/outer chưa được nêu ở REQ-09.

REQ-10 — Thanh toán

Tác nhân: Khách hàng đã đăng nhập, giỏ có hàng. Đầu vào: phương thức, cờ chấp thuận và khóa giao dịch. Căn cứ xác định kết quả đúng: COD = PENDING_COD; ONLINE được duyệt = PAID; bị từ chối = PAYMENT_DECLINED và 0 đơn. Rủi ro: tạo đơn/ghi nhận thu tiền không đúng khi từ chối.

Điểm cần làm rõ / giới hạn: Rollback tồn kho khi thanh toán bị từ chối thuộc ví dụ minh họa REQ-08; không tự gộp làm tiêu chí REQ-10.

REQ-15 — Tính nguyên tử khi trừ tồn kho

Tác nhân: Khách hàng. Điều kiện trước: giỏ nhiều SKU, tồn kho xác định. Căn cứ xác định kết quả đúng: OUT_OF_STOCK, tồn kho tất cả SKU không đổi và không có order nếu bất cứ SKU nào thiếu. Rủi ro: trừ SKU đủ hàng trước khi phát hiện SKU thiếu.

Điểm cần làm rõ / giới hạn: Mô hình chỉ kiểm tra xử lý tuần tự trong bộ nhớ; không chứng minh đồng thời hóa thực tế.

4. Điều kiện, tình huống kiểm thử và ma trận truy vết

Các điều kiện và tình huống được xác định trước khi đưa ra dữ liệu kiểm thử cụ thể. Bảng sau đối chiếu số lượng trường hợp thực hiện trên từng tuyến REQ/COND/TS.

Mã yêu cầu	Điều kiện	Tình huống	Ca kiểm thử liên kết
REQ-02	COND-CART-02	TS-CART-VALID	TC-CART-001
REQ-02	COND-CART-02	TS-CART-AGGREGATE	TC-CART-002, TC-CART-003
REQ-02	COND-CART-01	TS-CART-INVALID	TC-CART-004, TC-CART-005, TC-CART-006, TC-CART-007
REQ-07	COND-PROMO7-01	TS-PROMO7-INVALID	TC-PROMO7-001, TC-PROMO7-006
REQ-07	COND-PROMO7-01	TS-PROMO7-USED	TC-PROMO7-002, TC-PROMO7-005
REQ-07	COND-PROMO7-02	TS-PROMO7-SUCCESS	TC-PROMO7-003, TC-PROMO7-007
REQ-07	COND-PROMO7-02	TS-PROMO7-DECLINED	TC-PROMO7-004
REQ-09	COND-SHIP-01	TS-SHIP-INNER-THRESHOLD	TC-SHIP-001, TC-SHIP-002, TC-SHIP-003, TC-SHIP-006
REQ-09	COND-SHIP-02	TS-SHIP-OUTER	TC-SHIP-004, TC-SHIP-005
REQ-10	COND-PAY-01	TS-PAY-COD	TC-PAY-001, TC-PAY-002
REQ-10	COND-PAY-02	TS-PAY-ONLINE-APPROVED	TC-PAY-003, TC-PAY-004
REQ-10	COND-PAY-02	TS-PAY-ONLINE-DECLINED	TC-PAY-005, TC-PAY-006
REQ-15	COND-INV15-01	TS-INV15-REJECT	TC-INV15-001, TC-INV15-002, TC-INV15-003, TC-INV15-006
REQ-15	COND-INV15-02	TS-INV15-SUCCESS	TC-INV15-004, TC-INV15-005, TC-INV15-007

Tổng cộng 10 điều kiện, 14 tình huống và 33 ca kiểm thử; mỗi REQ có ít nhất hai điều kiện và hai tình huống. Kết quả thực thi bổ sung một mã lần chạy RUN cho từng TC trong tệp `reports/traceability_execution.csv`.



Ma trận REQ → COND → TS → TC

Dữ liệu nguyên gốc từ test_cases.csv · 33 dòng

test_case_id	requirement_id	condition_id	scenario_id	case_type
TC-CART-001	REQ-02	COND-CART-02	TS-CART-VALID	positive
TC-CART-002	REQ-02	COND-CART-02	TS-CART-AGGREGATE	positive
TC-CART-003	REQ-02	COND-CART-02	TS-CART-AGGREGATE	positive
TC-CART-004	REQ-02	COND-CART-01	TS-CART-INVALID	boundary
TC-CART-005	REQ-02	COND-CART-01	TS-CART-INVALID	negative
TC-CART-006	REQ-02	COND-CART-01	TS-CART-INVALID	negative
TC-CART-007	REQ-02	COND-CART-01	TS-CART-INVALID	negative
TC-PROMO7-001	REQ-07	COND-PROMO7-01	TS-PROMO7-INVALID	negative
TC-PROMO7-002	REQ-07	COND-PROMO7-01	TS-PROMO7-USED	negative
TC-PROMO7-003	REQ-07	COND-PROMO7-02	TS-PROMO7-SUCCESS	positive
TC-PROMO7-004	REQ-07	COND-PROMO7-02	TS-PROMO7-DECLINED	negative
TC-PROMO7-005	REQ-07	COND-PROMO7-01	TS-PROMO7-USED	negative
TC-PROMO7-006	REQ-07	COND-PROMO7-01	TS-PROMO7-INVALID	negative
TC-PROMO7-007	REQ-07	COND-PROMO7-02	TS-PROMO7-SUCCESS	positive
TC-SHIP-001	REQ-09	COND-SHIP-01	TS-SHIP-INNER-THRESHOLD	boundary
TC-SHIP-002	REQ-09	COND-SHIP-01	TS-SHIP-INNER-THRESHOLD	boundary
TC-SHIP-003	REQ-09	COND-SHIP-01	TS-SHIP-INNER-THRESHOLD	boundary
TC-SHIP-004	REQ-09	COND-SHIP-02	TS-SHIP-OUTER	boundary
TC-SHIP-005	REQ-09	COND-SHIP-02	TS-SHIP-OUTER	boundary
TC-SHIP-006	REQ-09	COND-SHIP-01	TS-SHIP-INNER-THRESHOLD	positive
TC-PAY-001	REQ-10	COND-PAY-01	TS-PAY-COD	positive
TC-PAY-002	REQ-10	COND-PAY-01	TS-PAY-COD	positive
TC-PAY-003	REQ-10	COND-PAY-02	TS-PAY-ONLINE-APPROVED	positive
TC-PAY-004	REQ-10	COND-PAY-02	TS-PAY-ONLINE-APPROVED	positive
TC-PAY-005	REQ-10	COND-PAY-02	TS-PAY-ONLINE-DECLINED	negative
TC-PAY-006	REQ-10	COND-PAY-02	TS-PAY-ONLINE-DECLINED	negative
TC-INV15-001	REQ-15	COND-INV15-01	TS-INV15-REJECT	negative
TC-INV15-002	REQ-15	COND-INV15-01	TS-INV15-REJECT	negative
TC-INV15-003	REQ-15	COND-INV15-01	TS-INV15-REJECT	negative
TC-INV15-004	REQ-15	COND-INV15-02	TS-INV15-SUCCESS	boundary
TC-INV15-005	REQ-15	COND-INV15-02	TS-INV15-SUCCESS	positive
TC-INV15-006	REQ-15	COND-INV15-01	TS-INV15-REJECT	boundary
TC-INV15-007	REQ-15	COND-INV15-02	TS-INV15-SUCCESS	positive

Bảng biểu diễn dữ liệu gốc; kiểm tra cùng file traceability_design.csv và traceability_execution.csv.

Hình 3: **E02 — Ma trận truy vết REQ → COND → TS → TC** — Hình minh chứng từ dữ liệu CSV thiết kế.

5. Các ca kiểm thử chi tiết và kết quả kỳ vọng

Bộ kiểm thử có 33 ca kiểm thử: 12 hợp lệ, 13 không hợp lệ, 8 giá trị biên; tổng số không hợp lệ/giá trị biên là 21, vượt mức tối thiểu. Tất cả ca kiểm thử đều có tên, độ ưu tiên, điều kiện trước, các bước thực hiện, dữ liệu khởi tạo, đầu vào, kỳ vọng và mã truy vết. Thực tế và trạng thái nằm trong báo cáo thực thi riêng.

Để dễ đối chiếu trên giấy, phần đầu vào và kỳ vọng được trình bày bằng lời; cấu trúc JSON nguyên gốc và các giá trị chính xác được lưu trong tệp test_cases.csv tại kho mã nguồn, không bị thay đổi.

Cấu trúc dữ liệu kiểm thử

`test_case_id`, `requirement_id`, `condition_id`, `scenario_id`, `title`, `priority`, `precondition`, `fixture_key`, `input_json`, `expected_type`, `expected_json`, `execution_status`, `actual_result`, `error_message`, `evidence_reference`

Ngoài các cột bắt buộc, dữ liệu có `case_type` và `test_steps` giúp phân loại kiểm thử và trình bày bước thao tác cụ thể. Các trường thực thi ở CSV đầu vào để trống; runner ghi thực tế và status sang các tệp trong `reports/`.

Nhóm REQ-02 — Số lượng nguyên dương và cộng dồn sản phẩm trùng mã.

TC-CART-001 — Thêm SKU với số lượng nguyên dương — HỢP LỆ · P2 · PASS

Truy vết: COND-CART-02 → TS-CART-VALID → RUN-TC-CART-001; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo giỏ trống với dữ liệu khởi tạo mới; 2. Thêm các SKU theo thứ tự trong đầu vào; 3. Đối chiếu ngoại lệ, các dòng giỏ và subtotal.

Dữ liệu đầu vào: Lần lượt thêm: A × 1.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Các dòng trong giỏ: A × 1; Tạm tính: 200.000.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-CART-002 — Cộng dồn SKU A sau hai lần thêm (2 và 3) — HỢP LỆ · P2 · PASS

Truy vết: COND-CART-02 → TS-CART-AGGREGATE → RUN-TC-CART-002; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo giỏ trống với dữ liệu khởi tạo mới; 2. Thêm các SKU theo thứ tự trong đầu vào; 3. Đối chiếu ngoại lệ, các dòng giỏ và subtotal.

Dữ liệu đầu vào: Lần lượt thêm: A × 2; A × 3.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Các dòng trong giỏ: A × 5; Tạm tính: 1.000.000.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-CART-003 — Hai SKU khác nhau tạo hai dòng giỏ hàng — HỢP LỆ · P2 · PASS

Truy vết: COND-CART-02 → TS-CART-AGGREGATE → RUN-TC-CART-003; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo giỏ trống với dữ liệu khởi tạo mới; 2. Thêm các SKU theo thứ tự trong đầu vào; 3. Đối chiếu ngoại lệ, các dòng giỏ và subtotal.

Dữ liệu đầu vào: Lần lượt thêm: A × 1; B × 2.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Các dòng trong giỏ: A × 1; B × 2; Tạm tính: 800.000.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-CART-004 — Từ chối số lượng bằng 0 — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-CART-01 → TS-CART-INVALID → RUN-TC-CART-004; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo giỏ trống với dữ liệu khởi tạo mới; 2. Thêm các SKU theo thứ tự trong đầu vào; 3. Đối chiếu ngoại lệ, các dòng giỏ và subtotal.

Dữ liệu đầu vào: Lần lượt thêm: A × 0.

Kết quả kỳ vọng: Ngoại lệ: ValueError:INVALID_QUANTITY; Các dòng trong giỏ: giỏ không có dòng hàng; Tạm tính: 0.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-CART-005 — Từ chối số lượng âm — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-CART-01 → TS-CART-INVALID → RUN-TC-CART-005; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo giỏ trống với dữ liệu khởi tạo mới; 2. Thêm các SKU theo thứ tự trong đầu vào; 3. Đối chiếu ngoại lệ, các dòng giỏ và subtotal.

Dữ liệu đầu vào: Lần lượt thêm: A × -1.

Kết quả kỳ vọng: Ngoại lệ: ValueError:INVALID_QUANTITY; Các dòng trong giỏ: giỏ không có dòng hàng; Tạm tính: 0.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-CART-006 — Từ chối số lượng không nguyên — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-CART-01 → TS-CART-INVALID → RUN-TC-CART-006; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo giỏ trống với dữ liệu khởi tạo mới; 2. Thêm các SKU theo thứ tự trong đầu vào; 3. Đối chiếu ngoại lệ, các dòng giỏ và subtotal.

Dữ liệu đầu vào: Lần lượt thêm: A × 1.5.

Kết quả kỳ vọng: Ngoại lệ: ValueError:INVALID_QUANTITY; Các dòng trong giỏ: giỏ không có dòng hàng; Tạm tính: 0.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-CART-007 — Từ chối số lượng ở dạng chuỗi — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-CART-01 → TS-CART-INVALID → RUN-TC-CART-007; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo giỏ trống với dữ liệu khởi tạo mới; 2. Thêm các SKU theo thứ tự trong đầu vào; 3. Đối chiếu ngoại lệ, các dòng giỏ và subtotal.

Dữ liệu đầu vào: Thêm SKU A với số lượng là chuỗi ký tự `"2"` (không phải số nguyên 2).

Kết quả kỳ vọng: Ngoại lệ: ValueError:INVALID_QUANTITY; Các dòng trong giỏ: giỏ không có dòng hàng; Tạm tính: 0.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

Nhóm REQ-07 — Kiểm tra tính hợp lệ của mã giảm giá và thời điểm ghi nhận đã sử dụng.

TC-PROMO7-001 — Từ chối mã giảm giá không hợp lệ — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-PROMO7-01 → TS-PROMO7-INVALID → RUN-TC-PROMO7-001; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Thiết lập mã giảm giá và khách hàng; 3. Gọi promotion hoặc checkout; 4. Đối chiếu used_codes, số đơn và ngoại lệ.

Dữ liệu đầu vào: Chế độ kiểm tra: áp dụng mã giảm giá; Mã giảm giá: FAKE.

Kết quả kỳ vọng: Ngoại lệ: ValueError:INVALID_VOUCHER; Số đơn hàng: 0; Mã giảm giá đã sử dụng: không có.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PROMO7-002 — Từ chối mã giảm giá đã sử dụng — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-PROMO7-01 → TS-PROMO7-USED → RUN-TC-PROMO7-002; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Thiết lập mã giảm giá và khách hàng; 3. Gọi promotion hoặc checkout; 4. Đối chiếu used_codes, số đơn và ngoại lệ.

Dữ liệu đầu vào: Chế độ kiểm tra: áp dụng mã giảm giá; Đánh dấu mã đã sử dụng khi khởi tạo: có; Mã giảm giá: SAVE10.

Kết quả kỳ vọng: Ngoại lệ: ValueError:VOUCHER_USED; Số đơn hàng: 0; Mã giảm giá đã sử dụng: SAVE10.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PROMO7-003 — Đơn COD thành công đánh dấu mã đã dùng — HỢP LỆ · P1 · PASS

Truy vết: COND-PROMO7-02 → TS-PROMO7-SUCCESS → RUN-TC-PROMO7-003; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Thiết lập mã giảm giá và khách hàng; 3. Gọi promotion hoặc checkout; 4. Đối chiếu used_codes, số đơn và ngoại lệ.

Dữ liệu đầu vào: Thanh toán được duyệt: có; Phương thức thanh toán: COD; Chế độ kiểm tra: đặt hàng; Mã giảm giá: SAVE10.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Số đơn hàng: 1; Trạng thái thanh toán: PENDING_COD; Mã giảm giá đã sử dụng: SAVE10.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PROMO7-004 — Thanh toán trực tuyến bị từ chối không tiêu thụ mã giảm giá — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-PROMO7-02 → TS-PROMO7-DECLINED → RUN-TC-PROMO7-004; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Thiết lập mã giảm giá và khách hàng; 3. Gọi promotion hoặc checkout; 4. Đối chiếu used_codes, số đơn và ngoại lệ.

Dữ liệu đầu vào: Thanh toán được duyệt: không; Phương thức thanh toán: ONLINE; Chế độ kiểm tra: đặt hàng; Mã giảm giá: SAVE10.

Kết quả kỳ vọng: Ngoại lệ: ValueError:PAYMENT_DECLINED; Số đơn hàng: 0; Trạng thái thanh toán: không có; Mã giảm giá đã sử dụng: không có.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PROMO7-005 — Không được dùng mã giảm giá lần hai sau đơn thành công — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-PROMO7-01 → TS-PROMO7-USED → RUN-TC-PROMO7-005; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Thiết lập mã giảm giá và khách hàng; 3. Gọi promotion hoặc checkout; 4. Đối chiếu used_codes, số đơn và ngoại lệ.

Dữ liệu đầu vào: Chế độ kiểm tra: dùng lại mã sau đơn thành công; Mã giảm giá: SAVE10.

Kết quả kỳ vọng: Ngoại lệ: ValueError:VOUCHER_USED; Số đơn hàng: 1; Trạng thái thanh toán: không có; Mã giảm giá đã sử dụng: SAVE10.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PROMO7-006 — Từ chối mã giảm giá đã ngừng hoạt động — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-PROMO7-01 → TS-PROMO7-INVALID → RUN-TC-PROMO7-006; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Thiết lập mã giảm giá và khách hàng; 3. Gọi promotion hoặc checkout; 4. Đối chiếu used_codes, số đơn và ngoại lệ.

Dữ liệu đầu vào: Đánh dấu mã không còn hiệu lực: có; Chế độ kiểm tra: áp dụng mã giảm giá; Mã giảm giá: SAVE10.

Kết quả kỳ vọng: Ngoại lệ: ValueError:INVALID_VOUCHER; Số đơn hàng: 0; Mã giảm giá đã sử dụng: không có.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PROMO7-007 — Đơn trực tuyến thành công đánh dấu mã đã dùng — HỢP LỆ · P1 · PASS

Truy vết: COND-PROMO7-02 → TS-PROMO7-SUCCESS → RUN-TC-PROMO7-007; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Thiết lập mã giảm giá và khách hàng; 3. Gọi promotion hoặc checkout; 4. Đối chiếu used_codes, số đơn và ngoại lệ.

Dữ liệu đầu vào: Thanh toán được duyệt: có; Phương thức thanh toán: ONLINE; Chế độ kiểm tra: đặt hàng; Mã giảm giá: SAVE10.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Số đơn hàng: 1; Trạng thái thanh toán: PAID; Mã giảm giá đã sử dụng: SAVE10.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

Nhóm REQ-09 — Phí vận chuyển nội thành và ngoại thành tại các mức giá biên.

TC-SHIP-001 — Nội thành ngay dưới ngưỡng miễn phí — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-SHIP-01 → TS-SHIP-INNER-THRESHOLD → RUN-TC-SHIP-001; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo Shipping; 2. Gọi calculate(Decimal subtotal, area); 3. So sánh phí vận chuyển.

Dữ liệu đầu vào: Khu vực giao hàng: nội thành; Tạm tính: 499.999.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Phí vận chuyển: 30.000.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-SHIP-002 — Nội thành đúng ngưỡng 500.000 — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-SHIP-01 → TS-SHIP-INNER-THRESHOLD → RUN-TC-SHIP-002; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo Shipping; 2. Gọi calculate(Decimal subtotal, area); 3. So sánh phí vận chuyển.

Dữ liệu đầu vào: Khu vực giao hàng: nội thành; Tạm tính: 500.000.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Phí vận chuyển: 0.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-SHIP-003 — Nội thành vượt ngưỡng miễn phí — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-SHIP-01 → TS-SHIP-INNER-THRESHOLD → RUN-TC-SHIP-003; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo Shipping; 2. Gọi calculate(Decimal subtotal, area); 3. So sánh phí vận chuyển.

Dữ liệu đầu vào: Khu vực giao hàng: nội thành; Tạm tính: 500.001.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Phí vận chuyển: 0.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-SHIP-004 — Ngoại thành dưới ngưỡng 500.000 — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-SHIP-02 → TS-SHIP-OUTER → RUN-TC-SHIP-004; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo Shipping; 2. Gọi calculate(Decimal subtotal, area); 3. So sánh phí vận chuyển.

Dữ liệu đầu vào: Khu vực giao hàng: ngoại thành; Tạm tính: 499.999.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Phí vận chuyển: 50.000.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-SHIP-005 — Ngoại thành tại ngưỡng 500.000 — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-SHIP-02 → TS-SHIP-OUTER → RUN-TC-SHIP-005; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo Shipping; 2. Gọi calculate(Decimal subtotal, area); 3. So sánh phí vận chuyển.

Dữ liệu đầu vào: Khu vực giao hàng: ngoại thành; Tạm tính: 500.000.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Phí vận chuyển: 50.000.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-SHIP-006 — Nội thành thông thường dưới ngưỡng — HỢP LỆ · P2 · PASS

Truy vết: COND-SHIP-01 → TS-SHIP-INNER-THRESHOLD → RUN-TC-SHIP-006; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo Shipping; 2. Gọi calculate(Decimal subtotal, area); 3. So sánh phí vận chuyển.

Dữ liệu đầu vào: Khu vực giao hàng: nội thành; Tạm tính: 300.000.

Kết quả kỳ vọng: Ngoại lệ: không phát sinh; Phí vận chuyển: 30.000.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

Nhóm REQ-10 — Trạng thái đơn hàng và giao dịch thanh toán được duyệt hoặc từ chối.

TC-PAY-001 — Đơn COD với giỏ 300.000 — HỢP LỆ · P2 · PASS

Truy vết: COND-PAY-01 → TS-PAY-COD → RUN-TC-PAY-001; **Dữ liệu khởi tạo:** 300k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 300.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Checkout bằng method và approved đã cho; 3. Kiểm tra trạng thái, số đơn và số charge.

Dữ liệu đầu vào: Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: không phát sinh; Số đơn hàng: 1; Trạng thái đơn hàng: PENDING_COD.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PAY-002 — Đơn COD với giỏ 600.000 — HỢP LỆ · P2 · PASS

Truy vết: COND-PAY-01 → TS-PAY-COD → RUN-TC-PAY-002; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Checkout bằng method và approved đã cho; 3. Kiểm tra trạng thái, số đơn và số charge.

Dữ liệu đầu vào: Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: không phát sinh; Số đơn hàng: 1; Trạng thái đơn hàng: PENDING_COD.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PAY-003 — Giao dịch ONLINE được duyệt với giỏ 300.000 — HỢP LỆ · P2 · PASS

Truy vết: COND-PAY-02 → TS-PAY-ONLINE-APPROVED → RUN-TC-PAY-003; **Dữ liệu khởi tạo:** 300k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 300.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Checkout bằng method và approved đã cho; 3. Kiểm tra trạng thái, số đơn và số charge.

Dữ liệu đầu vào: Thanh toán được duyệt: có; Phương thức thanh toán: ONLINE.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 1; Ngoại lệ: không phát sinh; Số đơn hàng: 1; Trạng thái đơn hàng: PAID.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PAY-004 — Giao dịch ONLINE được duyệt với giỏ 600.000 — HỢP LỆ · P2 · PASS

Truy vết: COND-PAY-02 → TS-PAY-ONLINE-APPROVED → RUN-TC-PAY-004; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Checkout bằng method và approved đã cho; 3. Kiểm tra trạng thái, số đơn và số charge.

Dữ liệu đầu vào: Thanh toán được duyệt: có; Phương thức thanh toán: ONLINE.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 1; Ngoại lệ: không phát sinh; Số đơn hàng: 1; Trạng thái đơn hàng: PAID.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PAY-005 — ONLINE bị từ chối, không tạo đơn với giỏ 300.000 — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-PAY-02 → TS-PAY-ONLINE-DECLINED → RUN-TC-PAY-005; **Dữ liệu khởi tạo:** 300k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 300.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Checkout bằng method và approved đã cho; 3. Kiểm tra trạng thái, số đơn và số charge.

Dữ liệu đầu vào: Thanh toán được duyệt: không; Phương thức thanh toán: ONLINE.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: ValueError:PAYMENT_DECLINED; Số đơn hàng: 0; Trạng thái đơn hàng: không có.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-PAY-006 — ONLINE bị từ chối, không tạo đơn với giỏ 600.000 — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-PAY-02 → TS-PAY-ONLINE-DECLINED → RUN-TC-PAY-006; **Dữ liệu khởi tạo:** 600k.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ 600.000.

Bước thực hiện: 1. Khởi tạo dữ liệu khởi tạo mới; 2. Checkout bằng method và approved đã cho; 3. Kiểm tra trạng thái, số đơn và số charge.

Dữ liệu đầu vào: Thanh toán được duyệt: không; Phương thức thanh toán: ONLINE.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: ValueError:PAYMENT_DECLINED; Số đơn hàng: 0; Trạng thái đơn hàng: không có.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

Nhóm REQ-15 — Từ chối toàn bộ đơn hàng nếu có sản phẩm không đủ tồn kho.

TC-INV15-001 — SKU A đủ, SKU B thiếu số lượng lớn — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-INV15-01 → TS-INV15-REJECT → RUN-TC-INV15-001; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo inventory mới; 2. Thêm tất cả SKU; 3. Ghi nhận tồn kho ban đầu; 4. Checkout; 5. So sánh tồn kho sau thao tác, số đơn và ngoại lệ.

Dữ liệu đầu vào: Lần lượt thêm: A × 2; B × 100; Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: ValueError:OUT_OF_STOCK; Số đơn hàng: 0; Tồn kho sau xử lý: A = 20, B = 15, C = 12.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-INV15-002 — SKU A thiếu trước, SKU B vẫn còn đủ — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-INV15-01 → TS-INV15-REJECT → RUN-TC-INV15-002; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo inventory mới; 2. Thêm tất cả SKU; 3. Ghi nhận tồn kho ban đầu; 4. Checkout; 5. So sánh tồn kho sau thao tác, số đơn và ngoại lệ.

Dữ liệu đầu vào: Lần lượt thêm: A × 21; B × 2; Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: ValueError:OUT_OF_STOCK; Số đơn hàng: 0; Tồn kho sau xử lý: A = 20, B = 15, C = 12.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-INV15-003 — SKU C thiếu sau khi SKU A đủ — KHÔNG HỢP LỆ · P1 · PASS

Truy vết: COND-INV15-01 → TS-INV15-REJECT → RUN-TC-INV15-003; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo inventory mới; 2. Thêm tất cả SKU; 3. Ghi nhận tồn kho ban đầu; 4. Checkout; 5. So sánh tồn kho sau thao tác, số đơn và ngoại lệ.

Dữ liệu đầu vào: Lần lượt thêm: A × 1; C × 13; Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: ValueError:OUT_OF_STOCK; Số đơn hàng: 0; Tồn kho sau xử lý: A = 20, B = 15, C = 12.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-INV15-004 — Đặt đúng bằng tồn kho A và B — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-INV15-02 → TS-INV15-SUCCESS → RUN-TC-INV15-004; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo inventory mới; 2. Thêm tất cả SKU; 3. Ghi nhận tồn kho ban đầu; 4. Checkout; 5. So sánh tồn kho sau thao tác, số đơn và ngoại lệ.

Dữ liệu đầu vào: Lần lượt thêm: A × 20; B × 15; Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: không phát sinh; Số đơn hàng: 1; Tồn kho sau xử lý: A = 0, B = 0, C = 12.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-INV15-005 — Đặt hàng thông thường với A và B — HỢP LỆ · P1 · PASS

Truy vết: COND-INV15-02 → TS-INV15-SUCCESS → RUN-TC-INV15-005; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo inventory mới; 2. Thêm tất cả SKU; 3. Ghi nhận tồn kho ban đầu; 4. Checkout; 5. So sánh tồn kho sau thao tác, số đơn và ngoại lệ.

Dữ liệu đầu vào: Lần lượt thêm: A × 1; B × 2; Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: không phát sinh; Số đơn hàng: 1; Tồn kho sau xử lý: A = 19, B = 13, C = 12.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-INV15-006 — SKU B thiếu đúng một sản phẩm — GIÁ TRỊ BIÊN · P1 · PASS

Truy vết: COND-INV15-01 → TS-INV15-REJECT → RUN-TC-INV15-006; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo inventory mới; 2. Thêm tất cả SKU; 3. Ghi nhận tồn kho ban đầu; 4. Checkout; 5. So sánh tồn kho sau thao tác, số đơn và ngoại lệ.

Dữ liệu đầu vào: Lần lượt thêm: B × 16; C × 1; Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: ValueError:OUT_OF_STOCK; Số đơn hàng: 0; Tồn kho sau xử lý: A = 20, B = 15, C = 12.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

TC-INV15-007 — Đặt hàng thông thường với C và A — HỢP LỆ · P1 · PASS

Truy vết: COND-INV15-02 → TS-INV15-SUCCESS → RUN-TC-INV15-007; **Dữ liệu khởi tạo:** empty.

Điều kiện trước: Dữ liệu khởi tạo riêng trên mã EduCommerce gốc; giỏ ban đầu trống.

Bước thực hiện: 1. Tạo inventory mới; 2. Thêm tất cả SKU; 3. Ghi nhận tồn kho ban đầu; 4. Checkout; 5. So sánh tồn kho sau thao tác, số đơn và ngoại lệ.

Dữ liệu đầu vào: Lần lượt thêm: C × 2; A × 1; Thanh toán được duyệt: có; Phương thức thanh toán: COD.

Kết quả kỳ vọng: Số lượt ghi nhận thu tiền: 0; Ngoại lệ: không phát sinh; Số đơn hàng: 1; Tồn kho sau xử lý: A = 19, B = 15, C = 10.

Kết quả thực thi: Đạt (PASS). Kết quả thực tế và thời gian của ca này được lưu trong báo cáo thực thi.

Fixture: tạo môi trường riêng cho mỗi test case

Trích nguyên văn từ test_runner.py · dòng 30–54

```
30  class CaseConfigurationError(Exception):
31      """Invalid test data/setup: cannot fairly judge application behavior."""
32
33
34  def fresh_state(fixture_key):
35      if fixture_key == 'empty':
36          app = Checkout()
37          return app, Customer('U_NEW'), Cart(app.catalog)
38      if fixture_key in ('300k', '499999', '500k', '600k', '1000k', '1500k'):
39          return fixture(fixture_key)
40      raise CaseConfigurationError(f'Unknown fixture_key: {fixture_key}')
41
42
43  def exception_name(error):
44      return f'{type(error).__name__}:{error}'
45
46
47  def cart_lines(cart):
48      return [{'sku': line.sku, 'quantity': line.quantity} for line in cart.lines]
49
50
51  def stock_snapshot(app):
52      return {sku: product.stock for sku, product in app.catalog.products.items()}
53
54
```

Mã nguồn: test_runner.py. Đối chiếu trực tiếp được trong repository.

Hình 4: **E03 — Hàm fresh_state() tạo dữ liệu khởi tạo độc lập** — Đoạn mã thực tế từ test_runner.py.

6. Cấu trúc chương trình chạy kiểm thử bằng Python

Chương trình chạy kiểm thử đọc các dòng dữ liệu CSV, kiểm tra đầu vào, tạo trạng thái độc lập cho từng ca và gọi chức năng phù hợp. Kết quả thực tế được so sánh với giá trị kỳ vọng để xác định PASS, FAIL hoặc BLOCKED. Sau cùng, chương trình xuất dữ liệu kết quả, ma trận truy vết và bảng tổng hợp dạng HTML.

Dữ liệu khởi tạo và tính độc lập của ca kiểm thử

`fresh_state(fixture_key)` tạo Checkout, Customer và Cart mới cho mỗi ca kiểm thử; dữ liệu khởi tạo hỗ trợ `empty, 300k, 499999, 500k, 600k, 1000k, 1500k`. Ví dụ giỏ rỗng khởi tạo tồn kho A=20, B=15, C=12. Cô lập trạng thái ngăn dữ liệu mã giảm giá, thanh toán và tồn kho ở ca kiểm thử trước làm lệch ca kiểm thử sau.

Đọc dữ liệu CSV và xử lý JSON

`pandas.read_csv` được dùng với `dtype=str`, `keep_default_na=False` và `encoding=utf-8-sig`; bộ đọc dữ liệu kiểm tra cột bắt buộc và từng ca kiểm thử xác thực ID, REQ, COND/TS cùng các trường JSON. Đầu vào thiếu hoặc phân tích cú pháp không hợp lệ được phân loại BLOCKED, không biến mất khỏi báo cáo. CSV rỗng/thiếu cấu trúc dữ liệu khiến phiên chạy dừng với mã lỗi.

Điều phối yêu cầu và đối chiếu kết quả

Mã yêu cầu	Lời gọi hoặc luồng	Trạng thái được xác minh
REQ-02	`Cart.add() / subtotal()`	lines, quantity, subtotal, exception

Mã yêu cầu	Lời gọi hoặc luồng	Trạng thái được xác minh
REQ-07	`Promotions.discount()` / `Checkout.place()`	used_codes, order_count, payment_status, exception
REQ-09	`Shipping.calculate()`	fee, exception
REQ-10	`Checkout.place()`	payment status, order_count, charge_count
REQ-15	`Checkout.place()` / `Inventory.reserve()`	stock_after của nhiều SKU, order_count, charge_count

`check_expected()` so sánh các trường được quy định trong `expected_json` với thực tế. Với kỳ vọng exception, chỉ đúng loại/thông báo dự kiến mới là PASS. Không đánh PASS chỉ vì có một order ID: những tác động phụ được mô tả trong tiêu chí chấp nhận cũng được đối chiếu.

Phân loại trạng thái và xuất báo cáo

PASS: đúng tất cả trường kỳ vọng; **FAIL**: thực tế trái kỳ vọng hoặc ứng dụng phát sinh lỗi không mong đợi; **BLOCKED**: dữ liệu, dữ liệu khởi tạo hay cấu hình kiểm thử không đủ điều kiện chạy. Runner ghi kết quả của mọi dòng kiểm thử và xuất năm tệp trong `reports/`.

Đọc CSV và xác thực dữ liệu đầu vào

Trích nguyên văn từ test_runner.py · dòng 144–183

```
144  def read_cases(path):
145      # dtype=str prevents Excel-like interpretation of test IDs; keep_default_na=False
146      # keeps blank execution fields blank. utf-8-sig accepts a Windows CSV BOM.
147      frame = pd.read_csv(path, dtype=str, keep_default_na=False, encoding='utf-8-sig')
148      missing = [column for column in REQUIRED_COLUMNS if column not in frame.columns]
149      if missing:
150          raise CaseConfigurationError('Missing CSV columns: ' + ', '.join(missing))
151      return frame.to_dict(orient='records')
152
153
154  def validate_case_inputs(req, data):
155      required = {
156          'REQ-02': ('actions',),
157          'REQ-07': ('mode', 'voucher'),
158          'REQ-09': ('subtotal', 'area'),
159          'REQ-10': ('method', 'approved'),
160          'REQ-15': ('actions', 'method', 'approved'),
161      }[req]
162      missing = [key for key in required if key not in data]
163      if missing:
164          raise CaseConfigurationError('Missing test input: ' + ', '.join(missing))
165      if req in ('REQ-02', 'REQ-15'):
166          actions = data['actions']
167          if not isinstance(actions, list) or any(
168              not isinstance(item, list) or len(item) != 2 for item in actions
169          ):
170              raise CaseConfigurationError('actions must be a list of [sku, quantity]')
171      if req == 'REQ-07':
172          if data['mode'] not in ('discount', 'checkout', 'reuse_after_success'):
173              raise CaseConfigurationError('Unknown promotion mode')
174          if data['mode'] == 'checkout':
175              missing = [key for key in ('method', 'approved') if key not in data]
176              if missing:
177                  raise CaseConfigurationError('Missing checkout input: ' + ', '.join(missing))
178      if req == 'REQ-09':
179          try:
180              Decimal(str(data['subtotal']))
181          except (ValueError, TypeError, ArithmeticError) as exc:
182              raise CaseConfigurationError('subtotal is not a valid decimal') from exc
183
```

Mã nguồn: test_runner.py. Đối chiếu trực tiếp được trong repository.

Hình 5: **E04 — Đọc CSV và xác thực dữ liệu** — Code sinh trực tiếp từ bản test_runner.py hiện hành.

Xử lý JSON và dữ liệu kiểm thử

Trích nguyên văn từ test_runner.py · dòng 185–211

```
185  def evaluate(row, repeated_ids):
186      start = time.perf_counter()
187      actual = {}
188      error = ''
189      status = 'BLOCKED'
190      tc_id = row['test_case_id'].strip()
191      try:
192          if not tc_id:
193              raise CaseConfigurationError('Empty test_case_id')
194          if tc_id in repeated_ids:
195              raise CaseConfigurationError(f'Duplicate test_case_id: {tc_id}')
196          req = row['requirement_id'].strip()
197          if SUPPORTED.get(req) != row['expected_type'].strip():
198              raise CaseConfigurationError('Unsupported requirement / expected_type combination')
199          if not row['condition_id'].strip() or not row['scenario_id'].strip():
200              raise CaseConfigurationError('Missing condition/scenario trace')
201          try:
202              inputs = json.loads(row['input_json'])
203              expected = json.loads(row['expected_json'])
204          except json.JSONDecodeError as exc:
205              raise CaseConfigurationError(f'Invalid input/expected JSON: {exc.msg}') from exc
206          if not isinstance(inputs, dict) or not isinstance(expected, dict):
207              raise CaseConfigurationError('input_json and expected_json must be JSON objects')
208          validate_case_inputs(req, inputs)
209          app, customer, cart = fresh_state(row['fixture_key'])
210          try:
211              actual = run_business_case(req, app, customer, cart, inputs)
```

Mã nguồn: test_runner.py. Đối chiếu trực tiếp được trong repository.

Hình 6: **E04 — Xử lý input_json và expected_json** — Phân biệt lỗi dữ liệu với lỗi nghiệp vụ.

Điều phối gọi hàm nghiệp vụ

Trích nguyên văn từ test_runner.py · dòng 55–88

```
55  def run_business_case(req_id, app, customer, cart, data):
56      """Invoke real EduCommerce functions and collect comparable return/state data."""
57      if req_id == 'REQ-02':
58          error = None
59          for sku, quantity in data['actions']:
60              try:
61                  cart.add(sku, quantity)
62              except ValueError as exc:
63                  error = exception_name(exc)
64                  break
65          return {'exception': error, 'lines': cart_lines(cart), 'subtotal': str(cart.subtotal())}
66
67      if req_id == 'REQ-07':
68          mode = data['mode']
69          voucher = data.get('voucher')
70          if data.get('setup_used'):
71              customer.used_codes.add(voucher)
72          if data.get('make_inactive'):
73              app.promotions.codes[voucher].active = False
74          error, status = None, None
75          try:
76              if mode == 'discount':
77                  app.promotions.discount(voucher, cart.subtotal(), customer)
78              elif mode == 'checkout':
79                  order = app.place(customer, cart, voucher=voucher,
80                                    method=data['method'], approved=data['approved'],
81                                    key='PROMO-ORDER-1')
82                  status = order.status
83              elif mode == 'reuse_after_success':
84                  app.place(customer, cart, voucher=voucher, key='PROMO-ORDER-1')
85                  app.place(customer, cart, voucher=voucher, key='PROMO-ORDER-2')
86              else:
87                  raise CaseConfigurationError(f'Unsupported promotion mode: {mode}')
88          except ValueError as exc:
```

Mã nguồn: test_runner.py. Đối chiếu trực tiếp được trong repository.

Hình 7: **E05 — Test dispatcher theo REQ** — Đoạn mã nhận requirement_id và gọi hàm tương ứng.

Điều phối và so sánh expected / actual

Trích nguyên văn từ test_runner.py · dòng 90–143

```
        return {'exception': error, 'used_codes': sorted(customer.used_codes),
                'order_count': len(app.orders), 'payment_status': status}

    if req_id == 'REQ-09':
        error, fee = None, None
        try:
            fee = str(Shipping().calculate(Decimal(str(data['subtotal'])), data['area']))
        except ValueError as exc:
            error = exception_name(exc)
        return {'exception': error, 'fee': fee}

    if req_id == 'REQ-10':
        error, status = None, None
        try:
            order = app.place(customer, cart, method=data['method'],
                              approved=data['approved'], key='PAY-ORDER-1')
            status = order.status
        except ValueError as exc:
            error = exception_name(exc)
        return {'exception': error, 'status': status,
                'order_count': len(app.orders), 'charge_count': len(app.payments.charges)}

    if req_id == 'REQ-15':
        for sku, quantity in data['actions']:
            cart.add(sku, quantity)
        error = None
        try:
            app.place(customer, cart, method=data['method'],
                      approved=data['approved'], key='INV-ORDER-1')
        except ValueError as exc:
            error = exception_name(exc)
        return {'exception': error, 'stock_after': stock_snapshot(app),
                'order_count': len(app.orders), 'charge_count': len(app.payments.charges)}

    raise CaseConfigurationError(f'Unsupported requirement: {req_id}')


def check_expected(expected, actual, prefix=''):
    """Compare the expected keys only; actual may include additional diagnostic fields."""
    differences = []
    if isinstance(expected, dict):
        if not isinstance(actual, dict):
            return [f'{prefix}: expected object, got {actual!r}']
        for key, value in expected.items():
            path = f'{prefix}.{key}' if prefix else key
            if key not in actual:
                differences.append(f'{path}: key missing')
            else:
                differences.extend(check_expected(value, actual[key], path))
    elif expected != actual:
        differences.append(f'{prefix}: expected={expected!r}; actual={actual!r}')
    return differences
```

Mã nguồn: test_runner.py. Đối chiếu trực tiếp được trong repository.

Hình 8: **E05 — So sánh kỳ vọng với thực tế** — Hình mã nguồn đối chiếu căn cứ xác định kết quả đúng và kiểm trạng thái.

Xác định trạng thái PASS / FAIL / BLOCKED

Trích nguyên văn từ test_runner.py · dòng 185–219

```
185  def evaluate(row, repeated_ids):
186      start = time.perf_counter()
187      actual = {}
188      error = ''
189      status = 'BLOCKED'
190      tc_id = row['test_case_id'].strip()
191      try:
192          if not tc_id:
193              raise CaseConfigurationError('Empty test_case_id')
194          if tc_id in repeated_ids:
195              raise CaseConfigurationError(f'Duplicate test_case_id: {tc_id}')
196          req = row['requirement_id'].strip()
197          if SUPPORTED.get(req) != row['expected_type'].strip():
198              raise CaseConfigurationError('Unsupported requirement / expected_type combination')
199          if not row['condition_id'].strip() or not row['scenario_id'].strip():
200              raise CaseConfigurationError('Missing condition/scenario trace')
201          try:
202              inputs = json.loads(row['input_json'])
203              expected = json.loads(row['expected_json'])
204          except json.JSONDecodeError as exc:
205              raise CaseConfigurationError(f'Invalid input/expected JSON: {exc.msg}') from exc
206          if not isinstance(inputs, dict) or not isinstance(expected, dict):
207              raise CaseConfigurationError('input_json and expected_json must be JSON objects')
208          validate_case_inputs(req, inputs)
209          app, customer, cart = fresh_state(row['fixture_key'])
210          try:
211              actual = run_business_case(req, app, customer, cart, inputs)
212          except CaseConfigurationError:
213              raise
214          except Exception as exc:
215              status = 'FAIL'
216              error = f'UNEXPECTED_APPLICATION_EXCEPTION: {type(exc).__name__}: {exc}'
217          else:
218              differences = check_expected(expected, actual)
219              status = 'FAIL' if differences else 'PASS'
```

Mã nguồn: test_runner.py. Đối chiếu trực tiếp được trong repository.

Hình 9: **E06 — Logic đánh giá PASS/FAIL/BLOCKED** — Kiểm tra trạng thái đúng phạm vi.

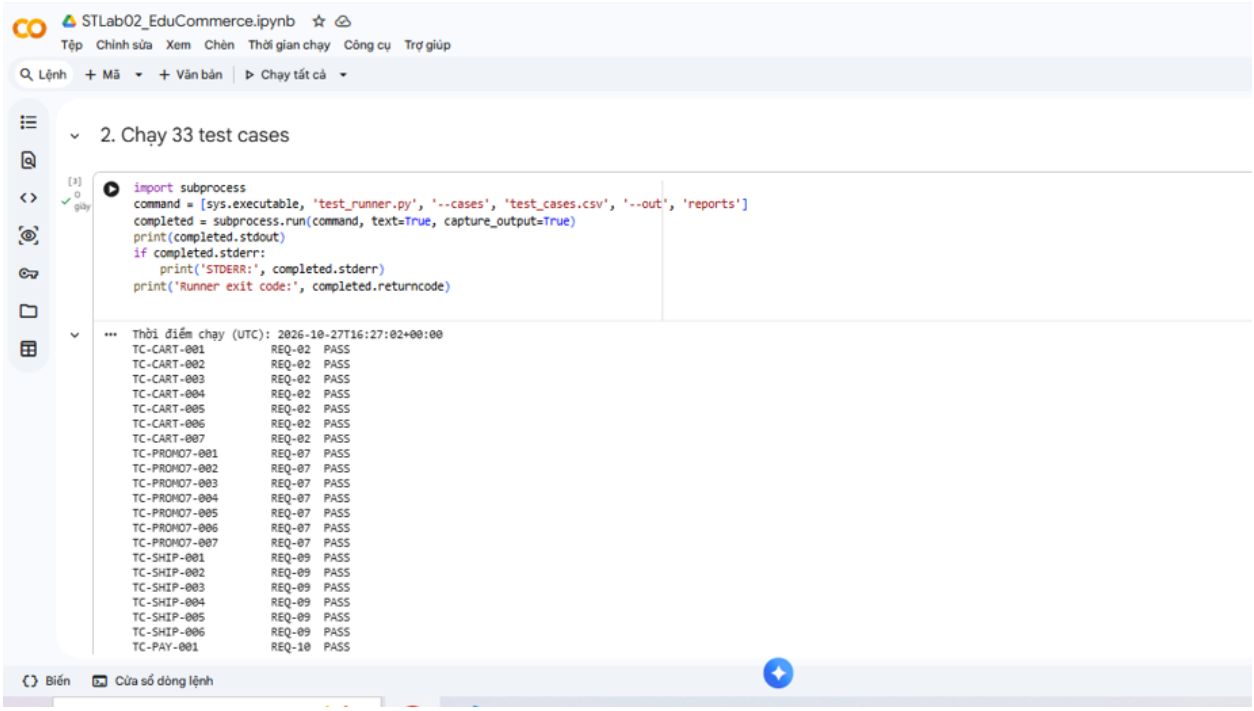
Xử lý ngoại lệ và ghi kết quả

Trích nguyên văn từ test_runner.py · dòng 212–248

```
212          except CaseConfigurationError:
213              raise
214          except Exception as exc:
215              status = 'FAIL'
216              error = f'UNEXPECTED_APPLICATION_EXCEPTION: {type(exc).__name__}: {exc}'
217          else:
218              differences = check_expected(expected, actual)
219              status = 'FAIL' if differences else 'PASS'
220              error = '; '.join(differences)
221      except (CaseConfigurationError, KeyError, TypeError, ValueError, IndexError) as exc:
222          status = 'BLOCKED'
223          error = f'TEST_SETUP: {type(exc).__name__}: {exc}'
224      except Exception as exc:
225          # An unexpected application error after valid data is FAIL, not a silent skip.
226          status = 'FAIL'
227          error = f'UNEXPECTED_EXCEPTION: {type(exc).__name__}: {exc}'
228      duration_ms = round((time.perf_counter() - start) * 1000, 3)
229      return {
230          'run_id': 'RUN-' + (tc_id or 'MISSING-ID'),
231          'test_case_id': tc_id,
232          'requirement_id': row['requirement_id'],
233          'condition_id': row['condition_id'],
234          'scenario_id': row['scenario_id'],
235          'title': row['title'],
236          'priority': row['priority'],
237          'execution_status': status,
238          'expected_json': row['expected_json'],
239          'actual_result': json.dumps(actual, ensure_ascii=False, sort_keys=True),
240          'error_message': error,
241          'evidence_reference': f'reports/test_execution.csv#{tc_id}',
242          'execution_time_ms': duration_ms,
243          'run_timestamp_utc': datetime.now(timezone.utc).isoformat(timespec='seconds'),
244      }
245
246
247  def export_csv(path, records, columns):
248      import csv
```

Mã nguồn: test_runner.py. Đối chiếu trực tiếp được trong repository.

Hình 10: **E06 — Nhánh xử lý ngoại lệ và lỗi dữ liệu** — Trích xuất từ runner hiện hành.

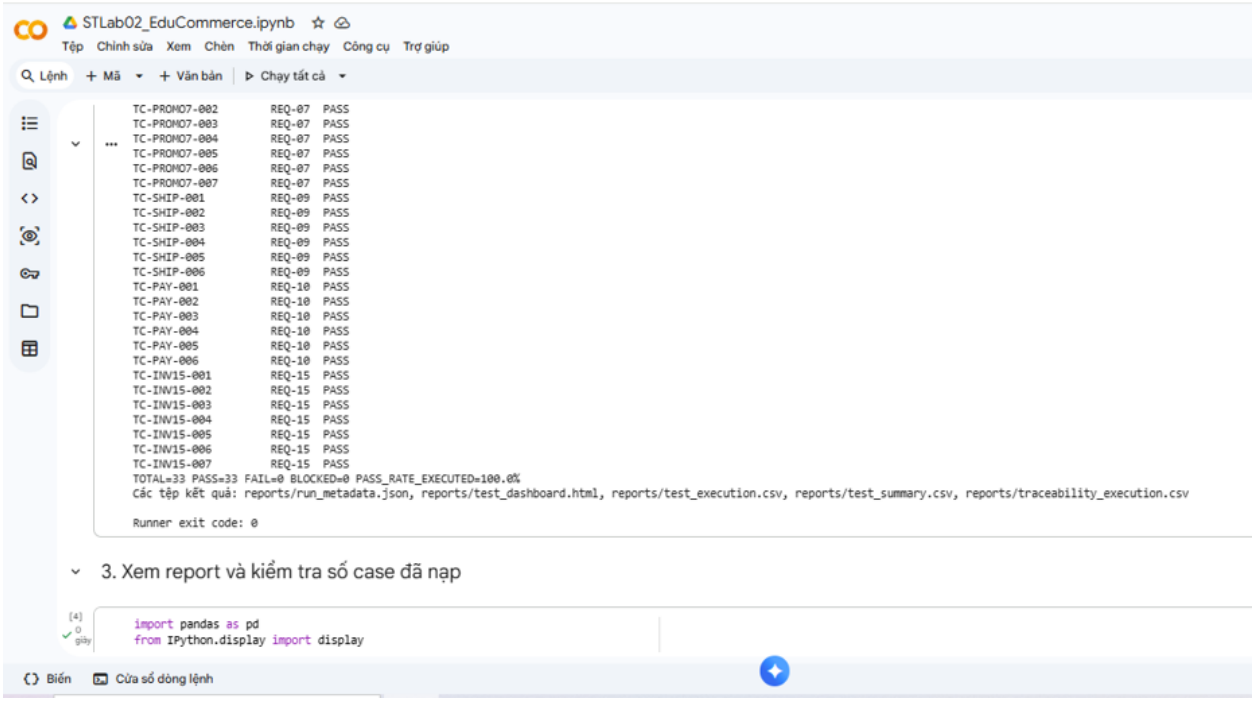


Hình 11: **E07 — Ảnh chạy Colab có timestamp** — Ảnh chụp thực tế do sinh viên cung cấp, bản sau sửa và kiểm tra lại.

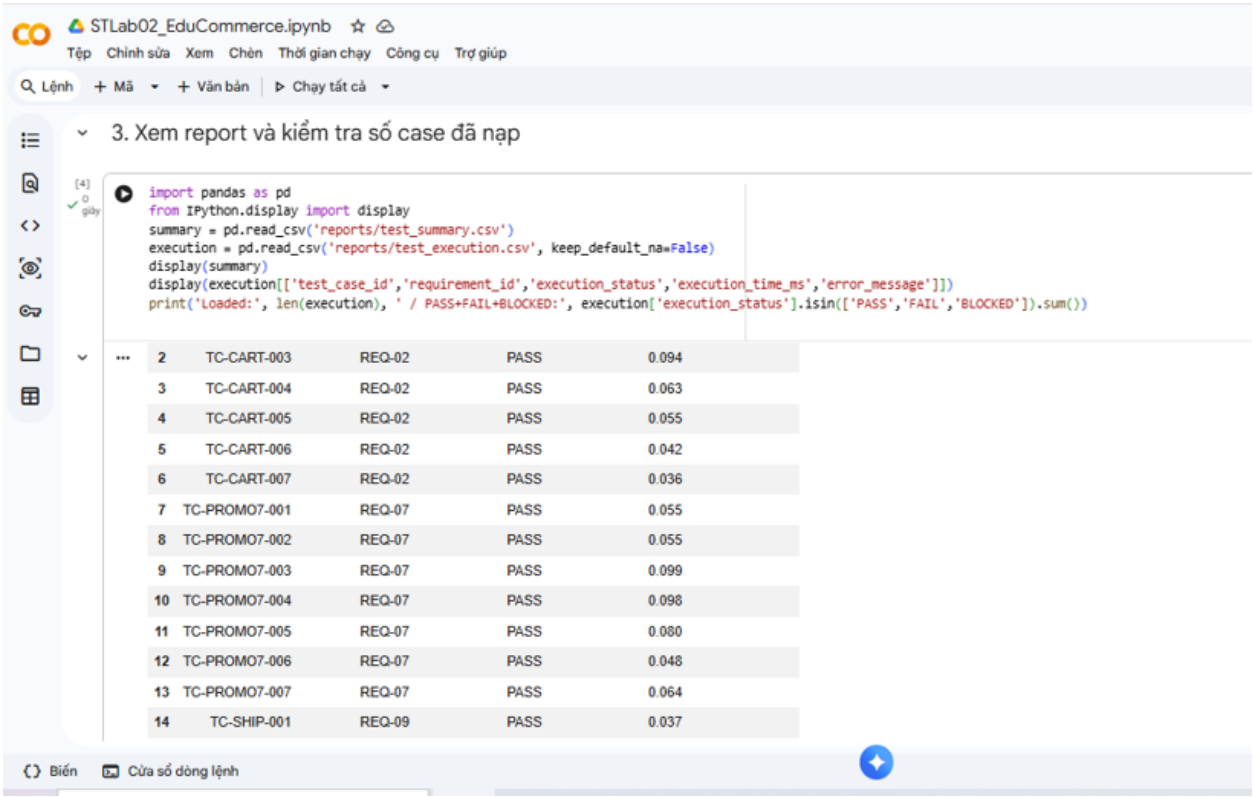
7. Thực thi trên Google Colab

Notebook `STLab02_EduCommerce.ipynb` đã được chạy bằng Python 3.13.16 trên Google Colab, với ba tệp tải lên cùng phiên bản: `ecommerce_app.py`, `test_runner.py`, `test_cases.csv`. Thời điểm ghi nhận theo UTC là `2026-10-27T16:27:02+00:00`, tương đương 23:27:02 cùng ngày tại Việt Nam.

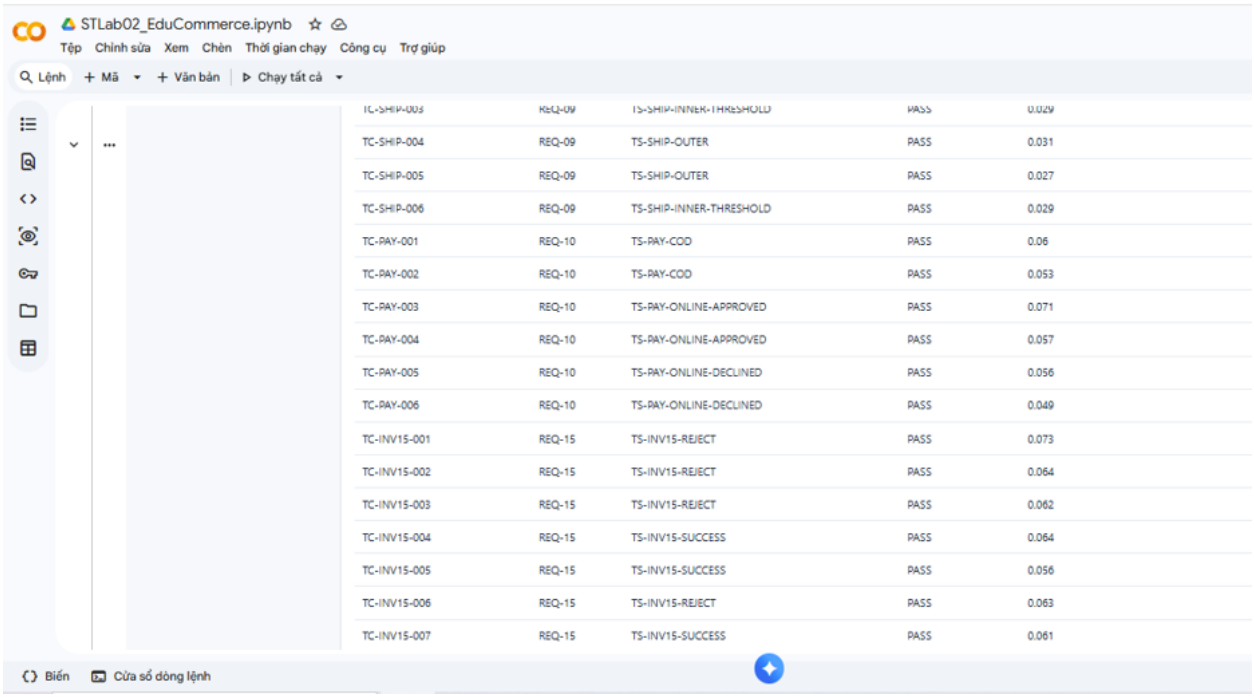
Thông số	Kết quả
Ca kiểm thử đã nạp	33
PASS / FAIL / BLOCKED	33 / 0 / 0
Tỷ lệ đạt trong số ca đã thực thi	100,0%
Runner exit code	0
Nguồn báo cáo	`reports/run_metadata.json`; `reports/test_execution.csv`



Hình 12: **E07 — Kết quả tổng kết 33 PASS, 0 FAIL, 0 BLOCKED** — Runner thoát với mã 0 trong phiên chạy hiện hành.



Hình 13: **E07 — Một phần bảng ca kiểm thử đã chạy trên Colab** — Các dòng kết quả được ghi nhận theo thứ tự trong cùng phiên chạy.



Hình 14: **E08 — Bảng tổng hợp HTML của lần chạy chính thức** — Hiển thị số lượng, trạng thái, biểu đồ theo yêu cầu và bảng chi tiết.

8. Báo cáo kết quả, bảng tổng hợp và độ bao phủ

Mã yêu cầu	Nạp	PASS	FAIL	BLOCKED
`REQ-02`	7	7	0	0
`REQ-07`	7	7	0	0
`REQ-09`	6	6	0	0
`REQ-10`	6	6	0	0
`REQ-15`	7	7	0	0
Tổng	**33**	**33**	**0**	**0**

Công thức đối chiếu: Loaded = PASS + FAIL + BLOCKED. Tỷ lệ PASS trong số kiểm thử được thực thi được tính PASS / (PASS + FAIL) khi mẫu số lớn hơn 0. BLOCKED luôn được báo riêng; không tự động bỏ các dòng không chạy được khỏi dữ liệu đầu vào.

Độ bao phủ và tính truy vết

Tất cả 33 ca kiểm thử trong `test_cases.csv` có mapping tới REQ/COND/TS và một mã lần chạy RUN tương ứng trong `reports/traceability_execution.csv`. Ma trận được dùng để kiểm tra sự đầy đủ của bộ kiểm thử được lựa chọn, không khẳng định bao phủ toàn bộ chức năng của EduCommerce.



Hình 15: **E08 — Bảng kết quả trong Colab** — Ảnh chụp thực tế từ notebook thực thi.

9. Phân tích lỗi và những rủi ro còn lại

Kết quả của lần chạy: 0 FAIL và 0 BLOCKED. Vì vậy không phát sinh defect ticket xác nhận đối với ứng dụng từ 33 ca kiểm thử này. Theo hướng dẫn của đề, báo cáo sử dụng nhánh trình bày evidence và hai rủi ro chưa kiểm thử.

Rủi ro chưa kiểm thử	Phạm vi / hệ quả	Hướng kiểm thử mở rộng
Hai checkout giữ cùng tồn kho đồng thời	Kiểm thử tuần tự in-memory chưa chứng minh tính nguyên tử khi hai request cùng cạnh tranh một SKU.	Bổ sung môi trường hỗ trợ xử lý đồng thời, hai thao tác checkout đồng thời và assertion bảo toàn tổng tồn kho.
Giá sản phẩm thay đổi giữa add-to-cart và checkout	Chưa có quy tắc chấp nhận xác định thời điểm chốt giá; căn cứ xác định kết quả đúng cho tổng tiền sẽ thiếu cơ sở nếu giá thay đổi.	BA/PO xác nhận chính sách chốt giá, sau đó tạo kiểm thử cho hai thời điểm và đối chiếu total.

Bộ runner cũng được kiểm chứng khả năng phân biệt lỗi dữ liệu dẫn tới BLOCKED và lỗi thực thi bất thường của ứng dụng dẫn tới FAIL. Những phép thử kỹ thuật này không phải defect của ứng dụng EduCommerce.

10. Giải quyết tám tình huống thực tế

Mỗi tình huống phân tích nguyên nhân/rủi ro, tệp hoặc hàm liên quan, thay đổi dự kiến, ca kiểm thử và kỳ vọng/evidence. Các ca kiểm thử mô tả trong phần này là đề xuất phục vụ bài giải tình huống; không tự tính là 33 trường hợp đã thực thi ở Phần 1, trừ trường hợp được định nghĩa và chạy trong bộ dữ liệu chính.

3.1. Hai SKU, một SKU thiếu tồn kho

Nguyên nhân và rủi ro: Checkout chứa A=2 (tồn kho 20) và B=100 (tồn kho 15). Một SKU thiếu hàng phải làm toàn bộ giao dịch thất bại. Nếu `Inventory.reserve` trừ A trước rồi mới phát hiện B thiếu, A sẽ bị trừ sai mặc dù không có đơn. Với mã nguồn gốc, `Inventory.reserve()` duyệt kiểm tra tồn kho của tất cả SKU rồi mới trừ; đó là cơ chế bảo vệ atomicity trong phạm vi mô phỏng tuần tự, không phải giao dịch cơ sở dữ liệu thật.

Tệp và hàm liên quan: `ecommerce_app.py` → `Inventory.reserve(lines)`; đường gọi `Checkout.place()`. **Thay đổi dự kiến nếu lỗi xuất hiện:** kiểm tra khả năng đáp ứng mọi dòng hàng trước khi thay đổi bất kỳ tồn kho nào; khi có failure khác sau khi reserve thì cần xử lý release/hoàn tác tương ứng theo REQ-08 (một yêu cầu mẫu riêng). Không tự sửa khi chưa tái hiện lỗi.

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng theo REQ-15	Minh chứng cần có
PS31-TC01	Dữ liệu khởi tạo mới, A=2, B=100; lưu tồn kho A20/B15; checkout COD	`OUT_OF_STOCK`; không có order; A20/B15 không đổi	Test đầu vào, exception, tồn kho before/after, code `reserve()`
PS31-TC02	Dữ liệu khởi tạo mới, A=20, B=15; checkout COD	Thành công, một order; A0/B0	Order và tồn kho before/after; ca kiểm thử dữ liệu biên

Không suy rộng: Mô hình này chưa chứng minh tính nguyên tử khi hai request chạy đồng thời.

3.2. Mã giảm giá chỉ được đánh dấu đã dùng khi đặt hàng thành công

Nguyên nhân và rủi ro: Thanh toán trực tuyến bị từ chối thì chưa có đơn thành công. Nếu `Customer.used_codes` đã thêm `SAVE10` trước đó, khách mất quyền dùng mã giảm giá dù giao dịch không thành công. `Checkout.place()` phải chỉ cập nhật mã giảm giá sau khi tạo đơn hợp lệ. Trong mã nguồn gốc, dòng `customer.used_codes.add(voucher)` nằm sau tạo `Order`, nhưng hậu quả về tồn kho khi payment failure thuộc REQ-08 và phải kiểm tra riêng.

Tệp và hàm liên quan: `ecommerce_app.py` → `Checkout.place()`, `Promotions.discount()` và `Payments.collect()`. **Thay đổi dự kiến khi có lỗi:** chỉ ghi used_codes sau commit thành công; bảo toàn used_codes ở mọi nhánh thất bại; nếu có giao dịch DB thực phải coi đây là cùng một giao dịch logic.

Ca kiểm thử đề xuất	Dữ liệu / các bước	Expected theo REQ-07	Minh chứng cần có
PS32-TC01	Dữ liệu khởi tạo 600k; `SAVE10`; ONLINE, `approved=False`; so used_codes trước/sau	`PAYMENT_DECLINED`; 0 order; mã giảm giá chưa used	Đầu vào, exception, used_codes, orders
PS32-TC02	Dữ liệu khởi tạo 600k; `SAVE10`; COD thành công; thử mã giảm giá lần hai với key khác	Đơn đầu thành công, mã đánh dấu used; lần hai `VOUCHER_USED`, không thêm order	Log hai thao tác, trạng thái mã, danh sách order

3.3. Mã thanh toán lạ `WALLET`

Nguyên nhân và rủi ro: Nếu checkout reserve hàng trước khi xác thực phương thức thanh toán, `WALLET` có thể làm tồn kho giảm rồi bị `Payments.collect()` từ chối, tạo side effect sai. Source gốc hiện gọi `Inventory.reserve()` trước khi kiểm tra method trong `Payments.collect()`; đó là một rủi ro cần tái hiện độc lập nếu giải quyết/triển

khai tình huống.

Tệp và hàm liên quan: `ecommerce_app.py` → `Checkout.place()`, `Payments.collect()`, `Inventory.reserve()`. **Thay đổi dự kiến:** validate `method in ('COD','ONLINE')` trước mọi biến đổi tồn kho/payment; hoặc bảo đảm hoàn tác toàn bộ nếu validation xảy ra sau reserve. Không tự bật hỗ trợ WALLET vì đề yêu cầu từ chối phương thức lạ (REQ-16).

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng	Minh chứng cần có
PS33-TC01	Dữ liệu khởi tạo 600k, `method='WALLET'`; lưu tồn kho và orders trước khi đặt	`INVALID_PAYMENT_METHOD`; 0 order; 0 charges; tồn kho giữ nguyên	Exception, trạng thái before/after, code trình tự
PS33-TC02	Cùng dữ liệu nhưng `method='COD'` trên dữ liệu khởi tạo mới	`PENDING_COD`; tạo một order; tồn kho trừ đúng số lượng	Log order, payments, tồn kho; đối chứng phương thức hợp lệ

3.4. Giá thay đổi giữa lúc thêm vào giỏ và lúc đặt hàng

Nguyên nhân và rủi ro: `Cart` chỉ giữ `(sku, quantity)`; `Cart.subtotal()` tra `Catalog.get(sku).price` tại thời điểm được gọi. Giá B tăng sau khi add thì subtotal lúc checkout có thể khác lúc khách bỏ hàng vào giỏ. REQ hiện tại chưa xác định thời điểm cố định giá; không được tự đưa ra kỳ vọng mới như một business rule đã được phê duyệt.

Tệp và hàm liên quan chịu ảnh hưởng: `Cart.add()`, `Cart.subtotal()`, `Catalog.get()`, `Checkout.place()`, và dữ liệu order. **Câu hỏi BA/PO:** Giá được cố định khi thêm vào giỏ, khi checkout hay khi xác nhận đặt hàng? Nếu giá đổi, người mua có cần được thông báo/đồng ý lại không? Có TTL hoặc price version không?

Thay đổi dự kiến theo câu trả lời: Nếu chốt tại add-to-cart thì lưu snapshot giá ở cart line; nếu chốt tại checkout thì dùng giá hiện hành, thông báo trước xác nhận. Chưa sửa code vì chưa có căn cứ chọn phương án.

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng có điều kiện	Minh chứng cần có
PS34-TC01	Add B qty=1 khi 300k, đổi B thành 350k rồi checkout	Chờ BA/PO: 300k nếu giá chốt ở add; 350k nếu giá chốt tại checkout, trước shipping	Price trước/sau, subtotal, order total, quyết định BA/PO
PS34-TC02	Add A+B, đổi giá B tại đúng ngưỡng miễn phí vận chuyển (subtotal từ 500k thành 450k)	Chờ BA/PO về giá chốt; shipping áp ngưỡng cho subtotal chính thức được chọn	Cart, giá, subtotal, shipping, chấp nhận clarification

Không kết luận PASS/FAIL trước khi làm rõ căn cứ xác định kết quả đúng.

3.5. Cùng SKU nhập hai lần

Nguyên nhân và rủi ro: Nếu `Cart.add()` tạo hai dòng riêng cho A thay vì cộng dồn, điều kiện tồn kho và subtotal có thể bị đánh giá sai. Source gốc tìm dòng theo SKU và tăng `quantity` nếu đã tồn tại. Lưu ý kiểm tra ranh giới số lượng: REQ-02 yêu cầu số nguyên dương; số tồn kho là điều kiện ở bước reserve chứ không tự động là giới hạn trên khi thêm vào giỏ.

Tệp và hàm liên quan: `Cart.add()`, `Cart.subtotal()`, `Inventory.reserve()`. **Thay đổi dự kiến nếu lỗi:** gộp theo SKU duy nhất; validate quantity trước khi thay đổi giỏ; không âm thầm biến quantity sai thành quantity hợp lệ.

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng	Minh chứng cần có
PS35-TC01	Add A=2, add A=3	Một dòng A qty=5; subtotal 1.000.000	`cart.lines`, subtotal, code `Cart.add()`
PS35-TC02	Add A=2 rồi add A=0	`INVALID_QUANTITY`; giỏ vẫn có một dòng A qty=2	Exception, cart before/after

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng	Minh chứng cần có
PS35-TC03	Add A=20 rồi add A=1; sau đó checkout	Cart có một dòng A qty=21; checkout `OUT_OF_STOCK`, không trừ tồn kho	Cart, lỗi checkout và tồn kho before/after

3.6. Các ca kiểm thử dựa trên dữ liệu phụ thuộc lẫn nhau

Nguyên nhân và rủi ro: Nếu nhiều ca kiểm thử dùng chung `Checkout`, `Catalog`, `Customer` hay `Cart`, ca kiểm thử trước có thể thay tồn kho, price, `used_codes`, `charges` hoặc `by_key`. Case sau phụ thuộc thứ tự chạy, gây false FAIL hoặc false PASS. Đây là lỗi thiết kế kiểm thử dữ liệu khởi tạo chứ chưa phải bug ứng dụng.

Tệp và hàm liên quan: `test_runner.py` → khởi tạo dữ liệu khởi tạo trước mỗi dòng; trong mã nguồn mẫu `ecommerce_app.py` → `fixture()`. **Thay đổi dự kiến:** tạo `Checkout`, `Customer`, `Cart` độc lập cho từng TC; không dùng service global hoặc giữ object từ lần chạy trước. Kiểm tra khi đảo thứ tự ca kiểm thử.

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng	Minh chứng cần có
PS36-TC01	TC-A đổi giá B thành 450k; TC-B mới dùng dữ liệu khởi tạo 300k	TC-B vẫn thấy giá B=300k; không kế thừa giá của TC-A	ID dữ liệu khởi tạo, before/after trạng thái, runner log
PS36-TC02	TC-A checkout A qty=3; TC-B mới kiểm tồn kho A ban đầu	TC-B nhìn thấy A tồn kho=20, không bị trừ xuống 17 từ TC-A	Tồn kho ở từng dữ liệu khởi tạo, chạy đảo thứ tự

3.7. Bảng thống kê hiển thị đạt dù thiếu ca kiểm thử

Nguyên nhân và rủi ro: CSV có 20 dòng, 3 dòng lỗi JSON và bị bỏ qua. Báo cáo chỉ còn 17 PASS. Dù `17/17 = 100%` cho các ca kiểm thử đã chạy, không được báo `20/20 PASS` hoặc che giấu 3 ca kiểm thử chưa chạy. Độ bao phủ theo dữ liệu gốc mới là 17/20; ba dòng còn lại phải được ghi BLOCKED (hoặc phân loại invalid data rõ ràng) thay vì biến mất.

Tệp và hàm liên quan: `test_runner.py` → CSV bộ đọc dữ liệu, validation, evaluator, reporter; HTML bảng tổng hợp. **Thay đổi dự kiến:** bắt lỗi phân tích ở cấp từng dòng, bảo toàn số dòng đã nạp, ghi trạng thái BLOCKED và lý do; tính `loaded = pass + fail + blocked`, báo riêng tỷ lệ thực thi và tỷ lệ đạt trong số ca kiểm thử thực thi.

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng	Minh chứng cần có
PS37-TC01	CSV 20 dòng, 17 hợp lệ, 3 `input_json` lỗi	loaded=20, PASS=17, BLOCKED=3, FAIL=0; không xóa ba dòng	Đầu vào CSV, báo cáo từng ca kiểm thử, tổng theo status
PS37-TC02	CSV 20 dòng đều hợp lệ và chạy PASS	loaded=20, PASS=20, FAIL=0, BLOCKED=0	CSV, thực thi log, HTML chart và bảng

Phân biệt công thức: `pass_rate_executed = PASS/(PASS+FAIL)` nếu mẫu số dương; `execution_coverage = (PASS+FAIL)/loaded`. Trường hợp thứ nhất lần lượt là 100% và 85%, không đồng nghĩa hoàn tất bài.

3.8. Đổi ngưỡng miễn phí vận chuyển 500k → 700k

Nguyên nhân và rủi ro: REQ-09 hiện miễn phí nội thành nếu subtotal >=500k. Đổi lên 700k ảnh hưởng trực tiếp quy tắc chấp nhận, giá trị biên tests, dữ liệu khởi tạo subtotals, `Shipping.calculate()` và cả order total ở các flow checkout. Đây là một giả định thay đổi yêu cầu trong bài Giải quyết tình huống, không phải lệnh thay đổi mã nguồn hay bộ kiểm thử hiện tại.

Tệp và hàm liên quan: `ecommerce_app.py` → `Shipping.calculate()`; các REQ/COND/TS/TC liên quan shipping; `test_cases.csv`; logic tổng tiền ở `Checkout.place()`. **Thay đổi dự kiến nếu được phê duyệt:** thay ngưỡng chính thức thành 700k (ưu tiên cấu hình/hằng số phù hợp mức bài), cập nhật kiểm thử data và kỳ vọng; chạy regression cả shipping và checkout total.

Ca kiểm thử đề xuất	Dữ liệu / các bước	Kết quả kỳ vọng nếu thay đổi được duyệt	Minh chứng cần có
PS38-TC01	`Shipping.calculate (699999,'inner')`	30000	Yêu cầu thay đổi, kiểm thử, output, mã nguồn
PS38-TC02	`Shipping.calculate (700000,'inner')`	0	Dữ liệu giá trị biên, kiểm thử, code
PS38-TC03	`Shipping.calculate (700001,'inner')`	0	Kiểm thử hồi quy giá trị biên

Ảnh hưởng cần kiểm thử hồi quy: mức 500k/600k trước đây được miễn phí nay chịu 30k; ngoại thành vẫn 50k. Phải cập nhật REQ-09, COND-SHIP-01, TS-SHIP-INNER-THRESHOLD, các TC-SHIP liên quan, dữ liệu khởi tạo và kiểm tra order total. Không cập nhật kỳ vọng của baseline 500k khi thay đổi chưa được phê duyệt.

11. Câu hỏi lý thuyết và bài tập phân tích mã

Phần này trả lời 20 câu hỏi lý thuyết và ba bài tập phân tích mã nguồn, liên hệ trực tiếp với EduCommerce. Các ví dụ dùng để giải thích kiến thức kiểm thử; không mặc định chúng đã được chạy trong 33 ca kiểm thử chính.

1. Phân biệt yêu cầu, điều kiện, tình huống, ca kiểm thử và lần thực thi?

Yêu cầu là quy tắc nghiệp vụ cần đáp ứng, chẳng hạn REQ-09 quy định mức phí giao hàng. Điều kiện kiểm thử xác định khía cạnh cần kiểm tra, như ngưỡng miễn phí. Tình huống kiểm thử mô tả hoàn cảnh cần thử: dưới, bằng hoặc trên ngưỡng. Ca kiểm thử bổ sung dữ liệu, bước thực hiện và kết quả kỳ vọng cụ thể. Lần thực thi ghi nhận kết quả thực tế, trạng thái và thời điểm chạy.

2. Căn cứ xác định kết quả đúng là gì? Vì sao không lấy kết quả thực tế làm kỳ vọng?

Căn cứ xác định kết quả đúng được lấy từ tiêu chí chấp nhận của yêu cầu và những quy tắc đã được làm rõ. Không thể lấy kết quả thực tế làm kỳ vọng, vì nếu chương trình đang xử lý sai dấu so sánh thì cách làm đó sẽ che giấu lỗi. Khi quy tắc chưa đủ rõ phải ghi nhận vấn đề cần xác minh, không tự đặt ra đáp án.

3. Phân lớp tương đương và giá trị biên trong tính giá, phí vận chuyển?

Phân lớp tương đương chia dữ liệu thành các nhóm được kỳ vọng xử lý giống nhau: giao hàng nội thành dưới ngưỡng, nội thành từ ngưỡng trở lên và ngoại thành. Phân tích giá trị biên tập trung vào ba mức 499.999, 500.000 và 500.001 đồng để phát hiện lỗi tại ranh giới. Các phép tính tiền dùng Decimal nhằm tránh sai số do biểu diễn số thực thông thường.

4. Ca hợp lệ, không hợp lệ và ca biên đối với Cart.add()?

Ca hợp lệ: thêm sản phẩm A với số lượng 2, giỏ nhận đúng hai sản phẩm. Ca không hợp lệ: thêm A với số lượng âm 1, hệ thống phải báo INVALID_QUANTITY và không đổi giỏ. Ca biên: số lượng 0 bị từ chối, trong khi số lượng 1 được chấp nhận. Ngoài kết quả trả về, cần kiểm tra trạng thái giỏ sau thao tác.

5. Dữ liệu khởi tạo là gì? Vì sao mỗi ca kiểm thử cần dữ liệu riêng?

Dữ liệu khởi tạo là trạng thái ứng dụng và dữ liệu mẫu trước khi chạy một ca kiểm thử: danh mục sản phẩm, tồn kho, giỏ hàng, khách hàng và các dịch vụ liên quan. Mỗi ca dùng trạng thái riêng để tránh việc đặt hàng, thay đổi tồn kho hoặc sử dụng mã giảm giá ở ca trước ảnh hưởng ca sau.

6. Cô lập ca kiểm thử khác gì khôi phục trạng thái sau mỗi lần chạy?

Tính độc lập nghĩa là kết quả của một ca kiểm thử không phụ thuộc thứ tự chạy các ca khác. Khởi tạo trạng thái mới trước mỗi ca là biện pháp phòng ngừa. Khôi phục trạng thái sau khi chạy cũng có thể áp dụng, nhưng dễ bị bỏ dở nếu quá trình kiểm thử dừng vì ngoại lệ. Với mô hình EduCommerce trong bộ nhớ, tạo trạng thái riêng giúp kiểm soát tốt hơn.

7. Khi dự kiến phát sinh ValueError, kết quả được xác định thế nào?

Nếu yêu cầu và dữ liệu của ca kiểm thử dự kiến phát sinh ValueError với mã INVALID_QUANTITY thì nhận đúng loại lỗi và thông điệp được tính là PASS. Không phát sinh lỗi, phát sinh lỗi khác hoặc thông điệp sai đều là FAIL. Nếu ngay từ đầu dữ liệu kiểm thử thiếu hoặc hỏng, ca đó được ghi BLOCKED, không nhầm với lỗi nghiệp vụ.

8. Vì sao kiểm tra giá trị trả về thôi là chưa đủ?

Chỉ kiểm tra việc tạo được mã đơn hàng chưa đủ để kết luận đúng. Một đơn hàng có thể đã được tạo nhưng tồn kho bị trừ sai hoặc phát sinh giao dịch thu tiền ngoài ý muốn. Vì vậy cần đối chiếu thêm tổng tiền, trạng thái

thanh toán, số đơn, tồn kho sau thao tác và tình trạng sử dụng mã giảm giá theo yêu cầu tương ứng.

9. Vì sao dùng Decimal thay cho float khi xử lý tiền?

Kiểu float có thể gây sai số khi biểu diễn tiền dưới dạng số nhị phân xấp xỉ. Decimal hỗ trợ tính toán thập phân chính xác theo giá trị đầu vào, phù hợp hơn với nghiệp vụ tiền tệ. Khi có chiết khấu hoặc làm tròn, chương trình vẫn phải tuân thủ chính sách làm tròn đã quy định.

10. Kiểm thử dựa trên dữ liệu là gì?

Kiểm thử dựa trên dữ liệu là cách sử dụng một chương trình chạy chung cho nhiều ca. Mỗi dòng CSV cung cấp mã ca kiểm thử, yêu cầu liên quan, dữ liệu khởi tạo, đầu vào và kết quả kỳ vọng. Chương trình đọc lần lượt từng dòng, thực hiện thao tác tương ứng, đối chiếu kết quả và ghi riêng trạng thái của ca đó.

11. Ma trận truy vết có tác dụng gì?

Ma trận truy vết liên kết yêu cầu với điều kiện, tình huống, ca kiểm thử và lần chạy. Nhờ đó, người kiểm thử có thể nhận biết yêu cầu chưa có kiểm tra, xác minh các ca đã thực hiện và xác định phần nào chịu ảnh hưởng khi yêu cầu thay đổi, chẳng hạn ngưỡng phí vận chuyển.

12. Độ ưu tiên của ca kiểm thử khác mức nghiêm trọng của lỗi thế nào?

Độ ưu tiên phản ánh mức cần thực hiện hoặc xử lý trước của một ca kiểm thử, xét theo thời gian và rủi ro. Mức nghiêm trọng phản ánh tác động của một lỗi khi xảy ra. Hai yếu tố liên quan nhưng không đồng nhất: một lỗi làm sai tiền có thể rất nghiêm trọng, còn thứ tự xử lý vẫn phụ thuộc xác suất và phạm vi ảnh hưởng.

13. Phân biệt kiểm thử đơn vị, tích hợp, hệ thống qua Checkout.place()?

Kiểm thử đơn vị tập trung vào một hàm nhỏ, ví dụ Shipping.calculate(). Kiểm thử tích hợp xem xét sự phối hợp giữa nhiều thành phần. Hàm Checkout.place() kết hợp giỏ hàng, khuyến mãi, vận chuyển, tồn kho và thanh toán nên phù hợp để đánh giá luồng tích hợp trong mô hình EduCommerce. Do không có giao diện và dịch vụ triển khai thực tế, không nên gọi đây là kiểm thử hệ thống hoàn chỉnh.

14. Các trạng thái chuyển đổi khi thanh toán COD và ONLINE?

Với phương thức COD, đơn hàng thành công có trạng thái PENDING_COD vì chưa thanh toán trực tuyến. Với ONLINE, chỉ giao dịch được chấp thuận mới có trạng thái PAID. Nếu thanh toán trực tuyến bị từ chối, chương trình phải phát sinh PAYMENT_DECLINED và không tạo đơn hàng. Cần kiểm tra cả số đơn và ghi nhận giao dịch thu tiền.

15. Tính không tạo tác động lặp trong đặt hàng là gì?

Tính không tạo tác động lặp có nghĩa là khi gửi lại cùng một yêu cầu với cùng khóa giao dịch, hệ thống không phát sinh thêm đơn hàng hay trừ tồn kho và ghi nhận thanh toán lần thứ hai. Vì vậy việc kiểm tra phải bao gồm mã đơn, số đơn, tồn kho và số giao dịch thu tiền trước và sau khi gửi lại.

16. Phân biệt lỗi ứng dụng và lỗi chương trình kiểm thử/dữ liệu khởi tạo?

Trước khi kết luận lỗi ứng dụng, cần kiểm tra quy tắc chấp nhận và dữ liệu kỳ vọng. Tiếp theo chạy lại bằng trạng thái mới, đầu vào tối giản, đối chiếu cách chọn hàm và so sánh kết quả. Nếu nguyên nhân là dữ liệu CSV sai hoặc chương trình chạy kiểm thử tạo trạng thái không đúng, đó là lỗi của quá trình kiểm thử, không phải lỗi của ứng dụng.

17. Bảng thống kê có thay thế được nhật ký thực thi gốc không?

Không. Bảng tổng hợp chỉ trình bày số liệu rút gọn nên có thể không thể hiện đầy đủ nguyên nhân một ca bị chặn hay lỗi xảy ra. Cần giữ tệp kết quả thực thi gốc với mã ca, kết quả kỳ vọng, kết quả thực tế, thông báo lỗi và thời điểm chạy. Bảng tổng hợp chỉ là phương tiện để đọc và so sánh nhanh.

18. Kiểm tra nhanh và kiểm thử hồi quy sau khi sửa lỗi?

Sau khi sửa lỗi, trước tiên kiểm tra nhanh chức năng vừa thay đổi bằng một số ca quan trọng. Sau đó chạy lại các ca có khả năng bị ảnh hưởng, đặc biệt các giá trị sát ranh giới 499.999, 500.000 và 500.001 đồng, các trường hợp ngoại thành và tổng tiền khi đặt hàng. Không sửa kết quả kỳ vọng chỉ để làm cho ca kiểm thử đạt.

19. Kho mã nguồn cần gì để có thể chạy lại trên Colab?

Kho mã nguồn cần có ứng dụng ecommerce_app.py, chương trình test_runner.py, bộ dữ liệu test_cases.csv, notebook Colab và README hướng dẫn chạy. Nên lưu kèm các tệp kết quả, bảng tổng hợp và thông tin phiên thực thi. Không đưa mật khẩu, mã truy cập hoặc dữ liệu bí mật lên kho công khai.

20. Nếu chỉ có thời gian chạy năm ca kiểm thử, nên ưu tiên ra sao?

Nên ưu tiên năm ca có rủi ro nghiệp vụ cao: đơn nhiều sản phẩm trong đó có một sản phẩm thiếu tồn kho; thanh toán trực tuyến bị từ chối không tạo đơn; mã giảm giá không bị mất lượt dùng khi giao dịch thất bại; phí vận chuyển tại đúng ngưỡng 500.000 đồng; và thêm cùng một mã sản phẩm hai lần phải cộng dồn đúng. Đây là lựa chọn ưu tiên khi thời gian hạn chế, không thay thế toàn bộ bộ kiểm thử.

Bài tập đọc và phân tích mã nguồn

A — Đối chiếu ngoại lệ theo kỳ vọng

Vấn đề: Nếu chương trình coi mọi ngoại lệ ValueError là FAIL thì một ca dự kiến phải từ chối dữ liệu không hợp lệ sẽ bị đánh giá sai.

Cách xử lý: Xác định trước ca kiểm thử kỳ vọng thành công hay kỳ vọng ngoại lệ. Khi có ngoại lệ dự kiến, đối chiếu cả loại và thông báo; khớp thì PASS, không khớp thì FAIL. Nếu dữ liệu ca kiểm thử không hợp lệ ngay từ lúc đọc, phân loại BLOCKED.

```
try:
    cart.add('A', 0)
    actual_error = None
except ValueError as error:
    actual_error = str(error)

assert actual_error == 'INVALID_QUANTITY'
```

Trường `expected_json` phải chứa căn cứ xác định kết quả đúng lỗi, chẳng hạn `{"exception":"ValueError:INVALID_QUANTITY"}`. Dữ liệu CSV phân tích cú pháp không hợp lệ hoặc fixture_key sai cần phân loại BLOCKED; đó không phải exception nghiệp vụ được mong đợi.

B — Kết quả đạt giả khi chỉ kiểm mã đơn hàng

Vấn đề: Kiểm tra duy nhất mã đơn hàng có thể tạo ra kết quả đạt giả, dù tồn kho hoặc tổng tiền bị xử lý sai. Phép kiểm tra phải bao gồm những trạng thái được nêu trong tiêu chí chấp nhận, không chỉ kiểm tra mã đơn khác rỗng.

Ví dụ dưới đây bổ sung kiểm tra tồn kho, tổng tiền, số đơn hàng và trạng thái thanh toán trong một tình huống đặt hàng. Các giá trị kỳ vọng phải được xác định từ quy tắc nghiệp vụ.

```
app, customer, cart = fixture('600k')  # A: 3 x 200000
before = app.catalog.get('A').stock
order = app.place(customer, cart, method='COD', key='CHECK-ONE')
assert order.order_id is not None
assert order.total == Decimal('600000')  # no voucher, free inner shipping
assert order.status == 'PENDING_COD'
assert app.catalog.get('A').stock == before - 3
assert len(app.orders) == 1
assert len(app.payments.charges) == 0
```

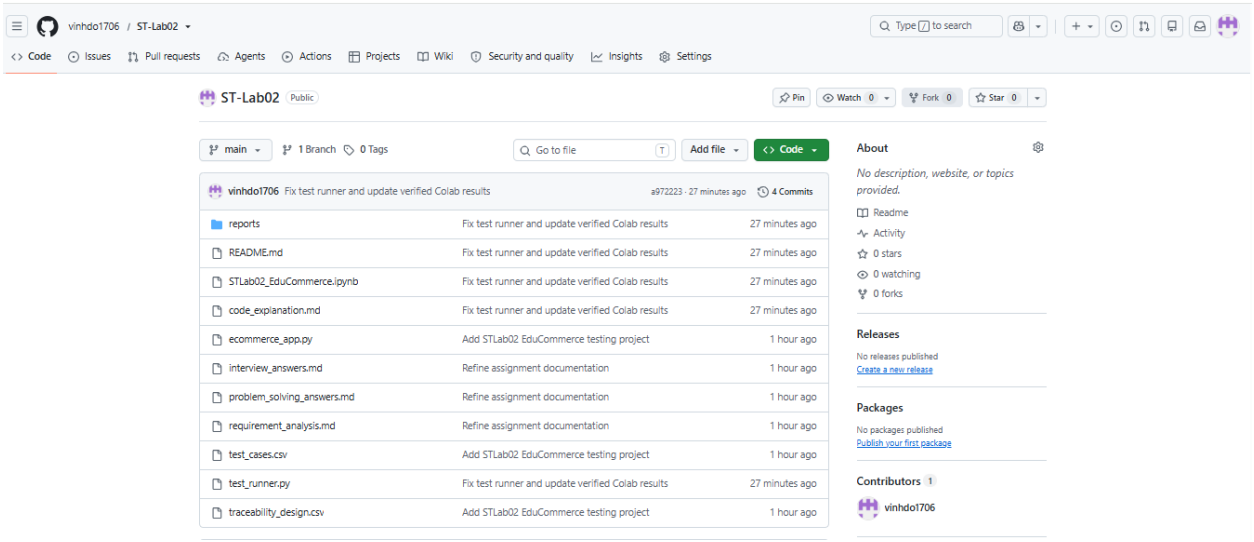
`fixture` và `Decimal` cần import từ `ecommerce_app/decimal` tương ứng trong code thực tế. Nếu có mã giảm giá, kỳ vọng total phải được tính từ rule discount/shipping đúng và dữ liệu của ca kiểm thử, không copy giá trị thực tế.

C — Dùng chung dữ liệu khởi tạo

Vấn đề: Khi nhiều ca dùng chung một đối tượng Checkout, tồn kho, mã giảm giá và danh sách đơn hàng có thể bị biến đổi qua các lần chạy. Kết quả khi đó phụ thuộc thứ tự thực thi.

Cách kiểm tra: Cho ca đầu thay đổi tồn kho hoặc giá, sau đó chạy ca tiếp theo. Đảo thứ tự hai ca và so sánh. Nếu kết quả kỳ vọng bị ảnh hưởng bởi ca trước, môi trường chưa được cô lập đúng.

Cách khắc phục: Khởi tạo đối tượng ứng dụng và dữ liệu riêng trước mỗi ca, thay vì dùng một đối tượng toàn cục. Chỉ giữ chung trạng thái giữa các bước bên trong cùng một ca kiểm thử khi tình huống thực sự cần.



Hình 16: **E10 — Repository Public, nhánh main và danh sách tệp** — Trích phần trang GitHub từ ảnh chụp gốc của sinh viên.

12. Tự đánh giá và tài liệu tham chiếu

Tiêu chí	Kết quả	Ghi chú
Chọn REQ không trùng bài mẫu	Đạt	5 REQ: 02, 07, 09, 10, 15
Điều kiện / tình huống	Đạt	10 điều kiện; 14 tình huống
Ca kiểm thử / Ca không hợp lệ và giá trị biên	Đạt	33 ca kiểm thử; 21 không hợp lệ/giá trị biên
Thực thi CSV bằng Python trên Colab	Đạt	33 PASS; 0 FAIL; 0 BLOCKED
Truy vết REQ → RUN	Đạt	33 mã lần chạy RUN tương ứng 33 ca kiểm thử
Báo cáo và bảng tổng hợp	Đạt	CSV, HTML bảng tổng hợp theo yêu cầu
Minh chứng E01–E10	Đạt theo trường hợp	E09 dùng nhánh không có FAIL + 2 rủi ro
Giải quyết tình huống	Có nội dung	8 tình huống theo đề
Câu hỏi lý thuyết	Có nội dung	20 câu hỏi + 3 Bài tập đọc mã
Giới hạn	Công bố rõ	Trong bộ nhớ, chưa kiểm xử lý đồng thời hoặc thay đổi giá

Kho mã nguồn và hướng dẫn chạy lại

Repository công khai: `https://github.com/vinhdo1706/ST-Lab02`. Tệp `README.md` mô tả cách mở notebook trên Google Colab, tải lên ba file đầu vào cùng phiên bản, chạy kiểm thử và xem các tệp `reports/`.

Tài liệu tham chiếu

1. *STLab02 — Test Case Execution (Requirement)*, đề bài do giảng viên cung cấp.
2. *STLab02 — Test Case Execution (Instruction)*, mô tả mô hình EduCommerce, tiêu chí chấp nhận và mã nguồn gốc.
3. Repository mã nguồn và kết quả thực thi: github.com/vinhdo1706/ST-Lab02.

Giới hạn kết luận: Tỷ lệ PASS 100% chỉ phản ánh 33 ca kiểm thử đối với năm yêu cầu đã chọn trên phiên bản và dữ liệu được ghi nhận. Không suy rộng thành xác nhận sản phẩm không còn lỗi.



4. Hướng dẫn chạy trên Google Colab

1. Truy cập Google Colab, chọn **File → Upload notebook** và mở `STLab02_EduCommerce.ipynb`.
2. Chạy cell đầu tiên để xem phiên bản Python và thư mục làm việc.
3. Tại cell **Upload source và CSV**, tải lên đồng thời ba file từ cùng phiên bản project: `ecommerce_app.py`, `test_runner.py`, `test_cases.csv`.
4. Chạy cell thực thi bộ test. Kiểm tra danh sách case, trạng thái PASS/FAIL/BLOCKED, thời gian chạy và thông báo lỗi nếu có.
5. Chạy các cell hiển thị bảng tổng hợp và HTML dashboard.
6. Chạy cell cuối để tải `STLab02_Colab_Execution.zip`, chứa các tệp kết quả trong `reports/`.
7. Lưu kết quả tải về và cập nhật đồng bộ các file trong thư mục `reports/` của project nếu thực hiện lần chạy mới.

Lưu ý: Môi trường chạy Colab có tính tạm thời. Khi thực thi lại, cần sử dụng notebook, mã nguồn và dữ liệu kiểm thử cùng phiên bản; lưu các tệp kết quả sau khi chạy.

Hình 17: **E10 — Hướng dẫn chạy trên Google Colab trong README** — Trích phần README từ ảnh chụp gốc của sinh viên.

Kết luận

Báo cáo trình bày quá trình phân tích và thực thi kiểm thử dựa trên yêu cầu cho năm chức năng đã lựa chọn trong EduCommerce. Bộ dữ liệu gồm 33 ca kiểm thử có mapping đến yêu cầu, điều kiện, tình huống và lần chạy; runner đọc CSV, khởi tạo dữ liệu khởi tạo riêng, so sánh kỳ vọng với thực tế và tạo bảng kết quả, ma trận truy vết cùng bảng tổng hợp HTML.

Trên Google Colab, 33 ca kiểm thử đã thực thi với kết quả 33 PASS, 0 FAIL và 0 BLOCKED. Các giới hạn kiểm thử còn lại được công bố ở phần rủi ro; kết quả không khẳng định sản phẩm không có lỗi trong mọi hoàn cảnh.

Mã nguồn, notebook, kiểm thử data, kết quả thực thi và hướng dẫn tái hiện được cung cấp tại repository:

`https://github.com/vinhdo1706/ST-Lab02`

HẾT